

**Diseño e Implementación de un Agente en Espacio de
Usuario**
**para la Gestión Dinámica de Frecuencia (DVFS) en
Sistemas**
**Heterogéneos mediante Modelos Ligeros de Machine
Learning**

Yeison Adrián Cáceres Torres

Ricardo Andrés Pérez Porras

Trabajo de Grado para Optar el Título de Ingeniero de Sistemas

Director

Gilberto Javier Díaz Toro

Doctor en Ingeniería

Universidad Industrial de Santander

Facultad de Ingenierías Físico-Mecánicas

Escuela de Ingeniería de Sistemas e Informática

Bucaramanga

2026

Agradecimientos

Resumen

Título: Diseño e Implementación de un Agente en Espacio de Usuario para la Gestión Dinámica de Frecuencia (DVFS) en Sistemas Heterogéneos mediante Modelos Ligeros de Machine Learning.

Autores: Yeison Adrián Cáceres Torres; Ricardo Andrés Pérez Porras.

Palabras Clave: DVFS; eficiencia energética; computación de alto rendimiento; modelo Roofline; telemetría microarquitectónica; aprendizaje automático supervisado.

Descripción:

El consumo energético constituye hoy una restricción estructural para el escalamiento de la computación de alto rendimiento (HPC), particularmente en nodos heterogéneos que integran CPU y aceleradores gráficos. El Escalado Dinámico de Voltaje y Frecuencia (DVFS) es el principal mecanismo de control disponible a nivel de hardware, pero su aprovechamiento efectivo depende de decidir, durante la ejecución, cuál punto operativo conviene según el comportamiento de la carga. Los gobernadores nativos del sistema operativo deciden a partir de la utilización agregada del procesador, una señal que no distingue si el rendimiento está limitado por la capacidad aritmética del núcleo o por la transferencia de datos desde memoria; en el segundo caso, sostener la frecuencia máxima incrementa el consumo sin una mejora proporcional del tiempo de ejecución.

Este trabajo propone el diseño, la implementación y la evaluación de un agente en espacio de usuario que infiere el régimen de ejecución de la aplicación a partir de telemetría microarquitectónica y traduce esa inferencia en decisiones discretas de DVFS sobre CPU y GPU, con el objetivo de optimizar el Producto Energía–Retardo (EDP) sin incurrir en una penalización severa del rendimiento ni en una sobrecarga de inferencia que anule el beneficio

energético.

El desarrollo se organiza en cuatro fases: caracterización de cargas y construcción del conjunto de datos; entrenamiento y selección de un clasificador supervisado ligero; implementación del servicio de control; y validación experimental frente a los gobernadores nativos de Linux. El presente documento reporta el instrumento de medición y el protocolo experimental contruidos durante la primera fase, cuyo aporte metodológico central es un procedimiento reproducible para etiquetar ventanas de ejecución como *compute-bound* o *memory-bound* mediante una calibración empírica del modelo Roofline sobre la plataforma de medición, con trazabilidad explícita de la calidad de cada observación.

Abstract

Title: Design and Implementation of a User-Space Agent for Dynamic Frequency Management (DVFS) in Heterogeneous Systems through Lightweight Machine Learning Models.

Authors: Yeison Adrián Cáceres Torres; Ricardo Andrés Pérez Porras.

Keywords: DVFS; energy efficiency; high performance computing; Roofline model; micro-architectural telemetry; supervised machine learning.

Description:

Energy consumption has become a structural constraint on the scaling of high performance computing (HPC), particularly on heterogeneous nodes that combine CPUs and graphics accelerators. Dynamic Voltage and Frequency Scaling (DVFS) is the main control mechanism available at the hardware level, but exploiting it effectively requires deciding, at run time, which operating point suits the current workload behaviour. Native operating system governors base that decision on aggregate processor utilisation, a signal that does not distinguish whether performance is limited by the arithmetic capability of the core or by data transfer from memory; in the latter case, sustaining maximum frequency raises power draw without a proportional improvement in execution time.

This work proposes the design, implementation and evaluation of a user-space agent that infers the execution regime of an application from microarchitectural telemetry and translates that inference into discrete DVFS decisions over CPU and GPU, aiming to optimise the Energy–Delay Product (EDP) without incurring a severe performance penalty or an inference overhead that cancels the energy benefit.

The work is organised in four phases: workload characterisation and dataset construction; training and selection of a lightweight supervised classifier; implementation of the control

service; and experimental validation against the native Linux governors. This document reports the measurement instrument and experimental protocol built during the first phase, whose central methodological contribution is a reproducible procedure for labelling execution windows as *compute-bound* or *memory-bound* through an empirical calibration of the Roofline model on the measurement platform, with explicit traceability of the quality of each observation.

Índice general

Agradecimientos	1
Resumen	2
Abstract	4
Introducción	10
Planteamiento y justificación del problema	11
Objetivos	13
1. Marco de referencia	15
1.1. Marco Conceptual	15
1.1.1. Termodinámica del silicio y potencia CMOS	15
1.1.2. Modelo Roofline y regímenes de limitación del rendimiento	16
1.1.3. Telemetría microarquitectónica	21
1.1.4. Espacios de ejecución: <i>user-space</i> y <i>kernel-space</i>	25
1.1.5. Interfaces estándar de observabilidad e instrumentación	26
1.1.6. Gestión dinámica de frecuencia en sistemas operativos: DVFS, P-states y <i>governors</i>	29
1.1.7. Cargas de trabajo: kernels, benchmarks y microbenchmarks	33
1.1.8. Aprendizaje automático supervisado ligero	35
1.1.9. Producto Energía–Retardo (EDP)	37
1.2. Marco Legal	38
1.3. Estado del Arte	38
2. Metodología	42
2.1. Enfoque metodológico	42
2.2. Diseño metodológico	43

2.2.1.	Población y muestra	43
2.2.2.	Plataforma experimental	43
2.2.3.	Instrumentos de recolección	44
2.2.4.	Variables observadas	45
2.3.	Fase 1: Recolección de información y caracterización de cargas	47
2.3.1.	Verificación previa de la plataforma	47
2.3.2.	Catálogo de cargas de trabajo	47
2.3.3.	Calibración y criterio de etiquetado	48
2.3.4.	Caracterización de intensidad operacional y calibración Roofline en GPU	49
2.3.5.	Dimensionamiento de las cargas GPU y verificación de invarianza de la intensidad operacional	51
2.3.6.	Matriz experimental y control de sesgos	52
2.3.7.	Post-procesamiento y control de calidad de los datos	53
2.3.8.	Medición directa de FLOPs por ventana y su validación empírica . .	56
2.3.9.	Reproducibilidad	58
2.4.	Fase 2: Desarrollo del modelo de aprendizaje automático	58
2.5.	Fase 3: Implementación del agente de control DVFS	59
2.6.	Fase 4: Validación experimental y análisis de resultados	60
2.7.	Aspectos éticos	60
3.	Resultados	62
3.1.	Descripción del conjunto de datos experimental	62
4.	Discusión	63
5.	Conclusiones	64
5.1.	Limitaciones	64
5.2.	Trabajo Futuro	64
	Repositorio y recursos complementarios	69

Índice de tablas

2.1. Características de la plataforma experimental.	44
2.2. Señales de telemetría capturadas por ventana de muestreo.	46
2.3. Composición del catálogo experimental de cargas de trabajo.	48
2.4. Estados de frecuencia de la matriz experimental.	53
2.5. Taxonomía de anotaciones de calidad por ventana de muestreo.	56

Índice de figuras

Introducción

En los sistemas de cómputo de alto rendimiento (HPC), el consumo energético se ha consolidado como una restricción crítica para el escalado sostenible, especialmente en arquitecturas heterogéneas CPU–GPU. En este contexto, el Escalado Dinámico de Voltaje y Frecuencia (DVFS) constituye el principal mecanismo de control a nivel de hardware, permitiendo ajustar los estados operativos de los procesadores para optimizar el compromiso entre consumo energético y tiempo de ejecución. Sin embargo, la efectividad de este mecanismo depende de la capacidad del sistema para adaptar dinámicamente la frecuencia al comportamiento cambiante de las aplicaciones, lo cual representa un desafío abierto en entornos HPC modernos.

En la literatura, este problema ha sido abordado principalmente mediante dos enfoques. Por un lado, los gobernadores de energía del sistema operativo, como `ondemand` o `schedutil`, emplean políticas reactivas basadas principalmente en la utilización del procesador, sin considerar la naturaleza microarquitectónica de la carga de trabajo, lo que limita su capacidad de adaptación a cargas de trabajo científicas dinámicas. Por otro lado, diversas propuestas han explorado estrategias heurísticas basadas en métricas microarquitectónicas, utilizando reglas deterministas para ajustar la frecuencia de operación. Aunque estos métodos presentan bajo *overhead*, su dependencia de umbrales estáticos y su limitada capacidad de generalización reducen su efectividad en escenarios donde las fases computacionales varían rápidamente.

A pesar de estos avances, persiste una limitación fundamental: la incapacidad de los enfoques actuales para capturar de forma robusta las transiciones dinámicas entre regímenes *compute-bound* y *memory-bound*, así como para adaptarse a la heterogeneidad CPU–GPU sin intervención manual o ajuste específico por carga de trabajo. Esta brecha evidencia la necesidad de mecanismos de control más flexibles que permitan inferir el régimen computacional de la aplicación en tiempo de ejecución y tomar decisiones de escalado de frecuencia fundamentadas en el perfil de ejecución.

En este contexto, el presente trabajo propone el diseño e implementación de un agente

en espacio de usuario basado en modelos ligeros de aprendizaje automático, capaz de clasificar en tiempo de ejecución las fases de una aplicación como *compute-bound* o *memory-bound* a partir de telemetría microarquitectónica. Con base en esta clasificación, el sistema ajusta dinámicamente la frecuencia de CPU y GPU con el objetivo de optimizar el Producto Energía–Retardo (EDP). Se espera que este enfoque permita mejorar la eficiencia energética en comparación con los gobernadores tradicionales de Linux, manteniendo una degradación controlada del rendimiento y un *overhead* mínimo asociado a la inferencia.

Ahora bien, antes de que un modelo de aprendizaje automático pueda decidir sobre la frecuencia de operación, es necesario disponer de un conjunto de datos etiquetado que asocie observaciones de telemetría con regímenes de ejecución. Construir ese conjunto no es un paso preparatorio trivial: exige resolver de forma explícita cómo se atribuye la telemetría al proceso medido, con qué resolución temporal se observa, bajo qué criterio se declara que una ventana de ejecución pertenece a un régimen u otro, y qué observaciones deben descartarse por no ser representativas. Estas decisiones condicionan la validez de todo lo que se construya después, razón por la cual el presente documento dedica una parte sustancial de su desarrollo metodológico a la construcción y verificación del instrumento de medición.

Planteamiento y justificación del problema

Existe una ineficiencia energética estructural en la operación de los sistemas de alto rendimiento (HPC), especialmente en infraestructuras que integran aceleradores gráficos (GPU) para ejecutar aplicaciones científicas intensivas. Cuando una aplicación entra en una fase donde el procesamiento depende principalmente de la transferencia de datos desde memoria, mantener la CPU o la GPU a su frecuencia máxima no mejora el rendimiento de manera proporcional. En estas condiciones, los núcleos de cómputo permanecen parcialmente inactivos esperando datos, lo que genera un consumo energético elevado sin un beneficio equivalente en tiempo de ejecución. Esta desalineación entre demanda computacional y frecuencia operativa ha motivado el estudio del escalado de frecuencia como mecanismo para mejorar la eficiencia energética en aplicaciones científicas [1] y en sistemas de gran escala donde la gestión de potencia impacta el comportamiento global del sistema [2].

El problema se agrava por la naturaleza multifásica de las aplicaciones HPC, cuyo perfil de ejecución cambia a lo largo del tiempo de ejecución. En particular, estas aplicaciones alternan entre fases donde el rendimiento está limitado por la capacidad de cómputo del procesador (*compute-bound*) y fases donde está restringido por la transferencia de datos desde memoria (*memory-bound*). Esta distinción es crítica porque, en el régimen *memory-*

bound, incrementos de frecuencia tienden a producir mejoras marginales en el tiempo de ejecución, mientras aumentan el consumo energético; en contraste, en fases *compute-bound* la frecuencia sí impacta el desempeño de forma más directa. Por ello, se han propuesto marcos de caracterización para identificar y clasificar estas fases en entornos HPC, con el fin de habilitar decisiones de control más informadas [3, 4].

A partir de este contexto, la gestión eficiente de frecuencia no depende únicamente de la disponibilidad del mecanismo DVFS en el hardware, sino de la capacidad del sistema para decidir, en tiempo de ejecución, cuál configuración resulta más conveniente según el comportamiento de la carga. Aunque se han propuesto mecanismos automatizados para seleccionar frecuencias óptimas de GPU con el fin de mejorar la eficiencia energética sin comprometer significativamente el desempeño [5], persiste una brecha en la implementación de estrategias que integren de forma coordinada componentes CPU–GPU y que operen durante la ejecución de la aplicación, considerando además el costo computacional asociado al propio proceso de decisión.

Esta brecha resulta especialmente relevante en entornos académicos y científicos, donde el consumo eléctrico incide directamente en los costos operativos, la disponibilidad efectiva de recursos y la sostenibilidad institucional. En este contexto, contar con un mecanismo capaz de observar el comportamiento de la aplicación, inferir su fase de ejecución y ajustar la frecuencia de operación sin intervenir el código fuente constituye una alternativa con valor técnico y práctico.

Desde la perspectiva internacional, la literatura reciente ha mostrado avances en la optimización energética mediante escalado de frecuencia, gestión de potencia y caracterización de cargas HPC [1, 2, 3, 4, 5]. Sin embargo, a nivel nacional y regional, este tipo de soluciones aún no se encuentra ampliamente implementado en infraestructuras académicas de cómputo científico, particularmente bajo esquemas que combinen telemetría de bajo nivel, modelos ligeros de aprendizaje automático y control en espacio de usuario. En consecuencia, el problema no solo es técnicamente relevante, sino también pertinente en términos de aplicabilidad local.

A partir de esta necesidad de optimización energética en sistemas heterogéneos, se formula la siguiente pregunta de investigación que servirá como eje de desarrollo del proyecto:

¿En qué medida el diseño e implementación de un agente en espacio de usuario, guiado por modelos ligeros de aprendizaje automático, permite ajustar la frecuencia de operación (DVFS) en sistemas heterogéneos CPU–GPU para reducir el consumo energético, sin que el costo de la inferencia computacional degrade el rendimiento global de la aplicación, en comparación con los gobernadores nativos de Linux?

Objetivos

Objetivo General

Caracterizar, diseñar, implementar y evaluar un agente en espacio de usuario basado en modelos ligeros de Aprendizaje Automático para el ajuste dinámico de frecuencia (DVFS) en sistemas heterogéneos CPU–GPU, con el propósito de maximizar la eficiencia energética en aplicaciones científicas sin inducir una penalización severa en su rendimiento global.

Objetivos Específicos

1. Caracterizar el comportamiento computacional y el consumo energético de cargas de trabajo representativas (intensivas en cómputo y en memoria) bajo distintos estados de frecuencia, recolectando telemetría de bajo nivel mediante contadores de rendimiento por hardware e interfaces de potencia estándar (Perf y RAPL para CPU y NVML para GPU).
2. Entrenar y validar modelos clásicos de Aprendizaje Automático (tales como Árboles de Decisión o Bosques Aleatorios) que sean capaces de clasificar, en tiempo de ejecución y con baja latencia de inferencia, las fases de ejecución de las aplicaciones basándose en la telemetría extraída.
3. Desarrollar un servicio de control (*daemon*) en el espacio de usuario que interactúe de forma asíncrona con el sistema operativo, leyendo el estado de los contadores de hardware, ejecutando la inferencia del modelo clasificador y aplicando políticas proactivas de DVFS a través de las interfaces estándar del sistema operativo, en función de la fase de ejecución inferida por el modelo.
4. Evaluar el impacto empírico del sistema propuesto mediante el cálculo del Producto Energía–Retardo (EDP), con el fin de determinar estadísticamente si el ahorro energético compensa la sobrecarga computacional (*overhead*) derivado de la inferencia del

modelo en comparación con los gobernadores nativos de Linux.

Capítulo 1

Marco de referencia

1.1. Marco Conceptual

El ajuste dinámico de frecuencia y voltaje para la optimización energética se fundamenta en la relación no lineal entre los parámetros de operación del hardware y la disipación de calor. A continuación, se definen los principios termodinámicos, arquitectónicos y de aprendizaje computacional que rigen el diseño de este agente de control en espacio de usuario, así como los conceptos de instrumentación y medición que sustentan la construcción del conjunto de datos experimental.

1.1.1. Termodinámica del silicio y potencia CMOS

La disipación de potencia en un circuito integrado moderno, como una CPU o una GPU basada en tecnología CMOS, puede descomponerse analíticamente en dos componentes principales: potencia estática, asociada a corrientes de fuga, y potencia dinámica, originada por la conmutación de cargas capacitivas en las compuertas lógicas. Dado que las técnicas de Escalado Dinámico de Voltaje y Frecuencia (DVFS) actúan sobre la frecuencia de operación y el voltaje de alimentación, su efecto principal recae sobre la potencia dinámica, la cual, para una compuerta CMOS, puede modelarse como:

$$P_{\text{dyn}} = \alpha \cdot C \cdot V^2 \cdot f \quad (1.1)$$

En donde α es el factor de actividad, C la capacitancia total del nodo de salida, V el voltaje de alimentación y f la frecuencia de operación. En circuitos más complejos, esta relación se extiende al chip completo, interpretando C como la capacitancia efectiva total

conmutada y α como un factor de actividad promedio [6].

Esta formulación constituye la base física que justifica el uso de técnicas DVFS. En los estados DVFS del hardware, la frecuencia de operación y el voltaje de alimentación se encuentran acoplados, ya que operar a frecuencias más altas suele requerir mayores voltajes para garantizar la integridad temporal y la estabilidad de las señales. En consecuencia, cuando una reducción de frecuencia permite también disminuir el voltaje, la potencia dinámica puede reducirse de forma significativa, debido a su dependencia lineal con la frecuencia y cuadrática con el voltaje. Esta relación constituye la base del compromiso energía–rendimiento en DVFS, ya que la reducción de voltaje y frecuencia disminuye la potencia dinámica, pero su efecto sobre el tiempo de ejecución depende del comportamiento computacional predominante de la aplicación.

1.1.2. Modelo Roofline y regímenes de limitación del rendimiento

El modelo Roofline proporciona un marco conceptual para analizar el rendimiento de una aplicación en función de su intensidad operacional y de los límites físicos del hardware. En su formulación clásica, el rendimiento alcanzable de un kernel puede expresarse como:

$$P = \text{mín} (P_{\text{pico}}, I \cdot BW_{\text{pico}}) \quad (1.2)$$

Donde P representa el rendimiento alcanzable, P_{pico} el rendimiento pico de cómputo del sistema, I la intensidad operacional y BW_{pico} el ancho de banda pico de memoria [7].

En el modelo original, la intensidad operacional se define como el número de operaciones por byte transferido entre la jerarquía de caché y la memoria principal, lo que permite relacionar directamente el comportamiento del kernel con la capacidad efectiva del subsistema de memoria. A partir de esta relación, el modelo establece una cota superior de rendimiento: si, para una determinada intensidad operacional, el límite viene dado por el término $I \cdot BW_{\text{pico}}$, el kernel se encuentra en un régimen limitado por memoria (*memory-bound*); por el contrario, si el límite viene dado por P_{pico} , el kernel se encuentra en un régimen limitado por cómputo (*compute-bound*) [7].

Punto de inflexión y calibración empírica del techo

La frontera entre ambos regímenes se ubica en el valor de intensidad operacional para el cual las dos ramas de la ecuación (1.2) se igualan. Este valor, denominado punto de inflexión o *ridge point*, se obtiene como:

$$I_{\text{ridge}} = \frac{P_{\text{pico}}}{BW_{\text{pico}}} \quad (1.3)$$

El punto de inflexión es el criterio operativo que permite convertir el modelo Roofline en una regla de decisión: una ventana de ejecución cuya intensidad operacional observada resulte inferior a I_{ridge} se encuentra en régimen *memory-bound*, y en régimen *compute-bound* en caso contrario.

Una precisión adicional es necesaria cuando el dispositivo caracterizado admite más de una precisión aritmética con rendimientos pico sustancialmente distintos entre sí, como ocurre en aceleradores gráficos modernos que ejecutan operaciones de simple y doble precisión a tasas diferentes: en ese caso, un único punto de inflexión calculado sobre un P_{pico} genérico no es representativo de ninguna de las dos precisiones, y la ecuación (1.3) debe evaluarse por separado para cada una, empleando en cada caso el P_{pico} correspondiente a esa precisión. El procedimiento de calibración por precisión, así como el criterio para evitar que el P_{pico} de referencia quede contaminado por una ruta de cómputo acelerada no representativa de la aritmética general del dispositivo, se describe en la sección 2.3.4.

Ahora bien, esta regla solo es aplicable si P_{pico} y BW_{pico} corresponden a la plataforma sobre la que efectivamente se mide. Los valores nominales publicados por el fabricante describen máximos teóricos que dependen de condiciones (número de núcleos activos, anchura de las unidades vectoriales, frecuencia sostenida, número de canales de memoria poblados) que rara vez coinciden con la configuración experimental concreta. Por esta razón, la determinación de ambos techos se aborda como una calibración empírica: se ejecutan cargas diseñadas para saturar cada uno de los dos límites por separado — una carga de ancho de banda puro para estimar BW_{pico} y una carga aritméticamente densa sobre datos residentes en caché para estimar P_{pico} — bajo las mismas condiciones de asignación de núcleos y afinidad que se emplearán en la campaña experimental. El valor de I_{ridge} resultante es, entonces, una propiedad de la plataforma tal como se la utiliza, y no una constante importada de una ficha técnica.

Estimación de la intensidad operacional observada

Aplicar el criterio anterior a una ventana de ejecución concreta exige estimar su intensidad operacional a partir de telemetría:

$$I = \frac{\text{FLOPs realizados en la ventana}}{\text{bytes movidos en la ventana}} \quad (1.4)$$

El denominador de la ecuación (1.4) no se observa de forma trivial mediante contadores

de propósito general, lo que introduce una consideración metodológica relevante desarrollada más abajo. El numerador, en cambio, sí se mide directamente por hardware en la plataforma experimental de este trabajo, mediante un procedimiento que se describe a continuación junto con la validación empírica que lo respalda.

La disponibilidad y la semántica de los eventos de hardware capaces de contabilizar operaciones de punto flotante no son uniformes entre microarquitecturas: dependiendo del procesador concreto, algunos de los eventos disponibles contabilizan instrucciones, microoperaciones ejecutadas o eventos especulativos en lugar de operaciones arquitectónicamente retiradas, lo que puede introducir sobreconteo o discrepancias respecto al volumen efectivo de trabajo de punto flotante — un problema documentado empíricamente en contadores de esta naturaleza [34]. Por esta razón, ningún evento de FLOPs se asume disponible ni correctamente mapeado por defecto: su encoding se verifica empíricamente para la microarquitectura concreta antes de usarse, siguiendo el mismo criterio ya aplicado en este instrumento para otros contadores de comportamiento no uniforme entre plataformas (sección 1.1.5).

Para la plataforma experimental de este trabajo (Intel Xeon Gold 5315Y, Ice Lake-SP), el evento `FP_ARITH_INST_RETIRED` (código de evento `0xC7`) contabiliza operaciones de punto flotante de doble precisión en cuatro sub-eventos, distinguidos por el ancho del registro vectorial que las ejecuta: escalar, y empaquetado de 128, 256 y 512 bits. Cada sub-evento representa un número distinto de operaciones por cuenta — 1, 2, 4 y 8 respectivamente, correspondiente al número de elementos de doble precisión que caben en cada ancho de registro — de modo que el total de FLOPs medidos en una ventana se obtiene como una suma ponderada:

$$\text{FLOPs}_i^{\text{HW}} = 1 \cdot \Delta_{\text{escalar}_i} + 2 \cdot \Delta_{128\text{B}_i} + 4 \cdot \Delta_{256\text{B}_i} + 8 \cdot \Delta_{512\text{B}_i} \quad (1.5)$$

El encoding crudo de los cuatro sub-eventos (sin mapeo genérico disponible en el kernel de este nodo) se resolvió mediante el mismo procedimiento ya empleado para otros contadores de esta plataforma (sección 1.1.5): contrastando contra la tabla de eventos validada de LIKWID para esta microarquitectura, que documenta explícitamente los grupos `FLOPS_DP` con esta misma ponderación [29], en vez de adivinarse a partir de documentación genérica. Herramientas de perfilado como LIKWID e Intel VTune [31], o interfaces de abstracción como PAPI [32], construyen sus propias métricas de FLOPs sobre estos mismos eventos, con soporte efectivo que continúa dependiendo del hardware subyacente y no de la interfaz.

Incorporar estos cuatro contadores exige presupuesto: junto con los seis eventos ya establecidos para las demás señales de la Tabla 2.2, agotan por completo el presupuesto de diez

contadores físicos simultáneos verificado empíricamente en esta plataforma (sección 1.1.3), sin margen para eventos adicionales. Antes de adoptar la medición directa como fuente principal de FLOPs, se verificó empíricamente — no se asumió — que esta combinación de diez contadores abre sin inducir multiplexación (razón tiempo-contando/tiempo-habilitado igual a 1 en los diez, en corridas de prueba dedicadas) y que los valores resultantes son físicamente plausibles. La validación consistió en contrastar la ecuación (1.5) contra el total analítico de FLOPs de un kernel de referencia (multiplicación de matrices densas, cuyo volumen de trabajo se conoce exactamente como $2 \cdot \text{iteraciones} \cdot n^3$): el error relativo fue de 0.29 % sin fijar afinidad de núcleos y de 0.30 % bajo la afinidad real de campaña (núcleos delegados fijados, hilos ajustados al mismo número de núcleos). Como segundo contraste, sobre un kernel de régimen opuesto (*memory-bound*, multi-hilo) el error relativo frente al total que el propio kernel reporta fue de 7.48 %, diferencia explicada por que el tiempo que dicho kernel cronometra internamente excluye su propia fase final de verificación de resultado — que también ejecuta operaciones de punto flotante — y no por un error del contador. Confirmada la viabilidad, la medición directa se incorporó como fuente principal de FLOPs por ventana en el instrumento, y se verificó a la escala completa de una campaña real (66 corridas, siete kernels, seis estados de frecuencia): de aproximadamente 1.29 millones de ventanas con delta válido, el 100 % obtuvo su valor de FLOPs por esta vía, sin que ninguna arrojara un contador degradado.

La ecuación (1.5) cubre únicamente operaciones de **doble** precisión: los cuatro sub-eventos de precisión simple del mismo evento de PMU (`FP_ARITH_INST_RETIRED`, umask `0x02/0x08/0x20/0x80`) no se abren, porque hacerlo excedería el presupuesto de diez contadores simultáneos ya agotado por completo. Esta decisión descansa en que las cargas del catálogo de CPU operan en doble precisión, coherente con la convención de las suites empleadas — NAS Parallel Benchmarks declara sus variables aritméticas en doble precisión por diseño, y la rutina de álgebra lineal invocada es `cblas_dgemm`, la variante de doble precisión de la interfaz BLAS, nunca su contraparte de precisión simple `cblas_sgemm` — pero esa convención no se dio por válida sin contrastarla: se repitió el procedimiento de validación anterior sustituyendo los cuatro sub-eventos de doble precisión por los de precisión simple, aplicado esta vez a los ocho kernels de CPU del catálogo experimental, no solo a los dos empleados en la validación principal. En los ocho casos, los cuatro contadores de precisión simple registraron cero o, cuando mucho, una o dos cuentas triviales en el sub-evento escalar — frente a órdenes de 10^9 a 10^{10} operaciones de doble precisión en la misma corrida — consistentes con ruido de arranque y no con trabajo aritmético del propio kernel. Con cobertura completa del catálogo de CPU, la precisión simple no constituye una fuente relevante de FLOPs en ninguna de sus cargas actuales; de incorporarse en el futuro un kernel nuevo,

su precisión debería contrastarse de la misma manera antes de asumir que la convención de doble precisión se extiende a él. Las cargas del catálogo de GPU con precisión simple declarada (`gpu_precision: fp32`, entre ellas varios kernels de la suite Rodinia) no están sujetas a esta restricción de presupuesto: su intensidad operacional se mide mediante Nsight Compute (sección 2.3.4), una vía de perfilado independiente que discrimina precisión de forma nativa y no comparte contadores de PMU de CPU con el mecanismo aquí descrito.

El procedimiento de prorratio empleado antes de esta validación — repartir un único total de FLOPs, reportado por el propio programa al finalizar la corrida, entre las ventanas en proporción a la fracción de instrucciones retiradas de cada una — se conserva en el instrumento como columna de auditoría y como mecanismo de respaldo para un nodo o una corrida donde la medición directa no esté disponible, pero deja de ser la fuente que alimenta la intensidad operacional siempre que la medición directa lo esté:

$$\widehat{\text{FLOPs}}_i = \text{FLOPs}_{\text{total}} \cdot \frac{\Delta \text{instrucciones}_i}{\text{instrucciones}_{\text{total}}} \quad (1.6)$$

El símbolo $\widehat{\text{FLOPs}}_i$ distingue este valor estimado del valor medido $\text{FLOPs}_i^{\text{HW}}$ de la ecuación (1.5). Su limitación de fondo — que una instrucción retirada y una operación de punto flotante no son la misma unidad, y que la ecuación (1.6) asume una densidad de FLOPs por instrucción homogénea dentro de la corrida, supuesto que no se sostiene necesariamente entre fases heterogéneas de una aplicación — ya no compromete la intensidad operacional reportada en este documento, puesto que la ecuación (1.6) solo se activa como respaldo, nunca como fuente principal en la plataforma y las campañas descritas aquí. El protocolo completo de esta validación, incluyendo el diseño experimental y los resultados agregados por kernel, se documenta en la sección 2.3.8.

En cuanto al denominador, el tráfico real hacia memoria principal se origina en el controlador de memoria, mientras que los contadores accesibles por proceso observan el comportamiento desde la perspectiva del núcleo. Una aproximación habitual consiste en estimar los bytes movidos como el producto entre el número de fallos del último nivel de caché y el tamaño de la línea de caché. Esta estimación es un *proxy*: no captura el tráfico generado por los mecanismos de precarga (*prefetch*) de hardware, que traen datos a la jerarquía antes de que un acceso de demanda los solicite, ni distingue el tráfico que se resuelve por reutilización en niveles intermedios. La magnitud de la discrepancia entre este *proxy* y el tráfico real depende del patrón de acceso del kernel y de la microarquitectura, por lo que constituye una fuente de error sistemático que debe declararse explícitamente en cualquier análisis que dependa del valor absoluto de la intensidad operacional. La cuantificación independiente de

este sesgo requiere acceso a los contadores del controlador de memoria, cuyas condiciones de disponibilidad se discuten en la sección 1.1.3.

1.1.3. Telemetría microarquitectónica

La aplicación selectiva de DVFS en sistemas heterogéneos requiere mecanismos de observación capaces de caracterizar, con baja intrusión, el comportamiento microarquitectónico de la carga de trabajo durante su ejecución. Para ello, los procesadores modernos incorporan unidades especializadas de monitorización y exponen interfaces estándar que permiten recolectar métricas relacionadas con la ejecución de instrucciones, el acceso a memoria y el consumo de potencia. Estas fuentes de telemetría constituyen la base empírica para inferir el estado computacional predominante de una aplicación y para cuantificar el efecto energético de las decisiones de control.

Unidades de monitorización de rendimiento (PMU) y contadores por hardware

Las Unidades de Monitorización de Rendimiento (*Performance Monitoring Units*, PMU) son componentes hardware integrados en los procesadores modernos para observar eventos microarquitectónicos de bajo nivel. Arquitectónicamente, una PMU está compuesta por registros de control y por contadores de monitorización de rendimiento (*Performance Monitoring Counters*, PMC), los cuales pueden emplearse para registrar eventos asociados al flujo de instrucciones, la jerarquía de memoria y otros aspectos relevantes del comportamiento del procesador [8]. En términos generales, estos contadores se clasifican en dos grupos: contadores fijos, destinados a eventos frecuentes como ciclos de reloj e instrucciones retiradas, y contadores programables o genéricos, configurables para medir distintos eventos del sistema [8].

A través de estos contadores es posible obtener instrumentación de baja sobrecarga y alta resolución para la caracterización de aplicaciones de computación de alto rendimiento. No obstante, aunque los procesadores soportan un conjunto amplio de eventos observables, el número de contadores físicos disponibles para medirlos simultáneamente es limitado, oscilando típicamente entre 1 y 4 PMC fijos y entre 4 y 8 PMC genéricos en arquitecturas modernas [8]. Esta restricción condiciona la selección de métricas y obliga a priorizar aquellos eventos más representativos para la caracterización del comportamiento de la aplicación. En consecuencia, las PMU constituyen una fuente de información más rica que las métricas globales tradicionalmente empleadas por los gobernadores del sistema operativo, pero su aprovechamiento exige criterios adecuados de selección y organización de eventos.

Multiplexación de contadores y escalado de medidas

Cuando el número de eventos solicitados excede el número de contadores físicos disponibles, el sistema operativo no puede medirlos todos de forma simultánea. La solución habitual es la multiplexación: el kernel rota periódicamente los eventos sobre los contadores disponibles, de modo que cada evento se mide durante una fracción del intervalo total. Para preservar la interpretabilidad de la medida, la infraestructura de monitorización acompaña cada lectura con dos magnitudes temporales: el tiempo durante el cual el evento estuvo habilitado y el tiempo durante el cual estuvo efectivamente contando. La razón entre ambos permite escalar linealmente el conteo observado hacia una estimación del conteo que se habría obtenido con medición continua.

Este escalado es una extrapolación, no una medida exacta: asume que la tasa del evento durante los intervalos no observados es equivalente a la de los intervalos observados, supuesto que se debilita cuando el comportamiento de la aplicación varía rápidamente dentro de la ventana de muestreo, que es precisamente el escenario de interés en aplicaciones multifásicas. La selección de eventos, por tanto, no es únicamente una decisión de cobertura informativa, sino también una decisión sobre la calidad de la medida: mantener el conjunto de eventos dentro del presupuesto de contadores físicos de la plataforma evita introducir error de extrapolación en la variable que se pretende observar. La problemática general de asignar eficientemente eventos a contadores en distintas plataformas ha sido abordada explícitamente en la literatura [8].

Eventos genéricos y eventos crudos

Las interfaces de monitorización modernas ofrecen dos formas de solicitar un evento. Los eventos genéricos constituyen una abstracción portable: el programa solicita un concepto (instrucciones retiradas, ciclos, fallos de caché) y el kernel lo traduce a la codificación concreta que corresponde a la microarquitectura sobre la que se ejecuta. Los eventos crudos, en cambio, se especifican mediante la codificación nativa del procesador, típicamente compuesta por un selector de evento, una máscara de sub-evento y, en algunos casos, calificadores adicionales que condicionan cuándo se incrementa el contador.

La abstracción genérica es preferible por razones de portabilidad y de verificabilidad, pero presenta una limitación práctica: depende de que el kernel disponga de una entrada de traducción para el modelo de procesador concreto. Cuando dicha entrada no existe — situación que puede presentarse en combinaciones de microarquitecturas recientes con versiones de kernel que no incorporan su tabla de eventos — la solicitud del evento genérico falla, aun cuando el contador físico subyacente exista en el hardware. En esos casos, acceder al evento

requiere recurrir a la codificación cruda, lo que traslada al instrumento la responsabilidad de garantizar que dicha codificación es correcta para esa microarquitectura específica. Esta responsabilidad no es menor: una codificación incompleta puede abrir el contador sin error aparente y producir valores carentes de sentido físico, un modo de fallo más peligroso que un error explícito, porque no se manifiesta hasta que se examina la coherencia de los datos.

Contadores de núcleo y contadores de *uncore*

No todos los contadores de un procesador observan el mismo dominio. Los contadores asociados al núcleo observan eventos atribuibles a la ejecución de instrucciones sobre ese núcleo, y por tanto pueden asociarse a un proceso concreto. Existe además un conjunto de unidades de monitorización situadas fuera de los núcleos — comúnmente denominadas *uncore* — que observan recursos compartidos del zócalo, entre ellos el controlador de memoria integrado, la caché de último nivel distribuida y las interconexiones internas.

Esta distinción es central para la medición del tráfico de memoria. Los contadores del controlador de memoria observan las transacciones que efectivamente alcanzan la memoria principal, con independencia de si fueron originadas por un acceso de demanda o por un mecanismo de precarga, por lo que constituyen la referencia adecuada para validar la estimación de bytes movidos descrita en la sección 1.1.2. Sin embargo, precisamente porque observan recursos compartidos y no procesos individuales, estos eventos son de alcance de sistema y no pueden atribuirse a un proceso específico, lo que los sitúa en una categoría de privilegio distinta a la de los contadores por proceso, tal como se detalla en la sección 1.1.5.

Métricas microarquitectónicas de eficiencia

La caracterización de los regímenes *compute-bound* y *memory-bound* en sistemas reales no suele derivarse de una observación directa del estado interno de la aplicación, sino de señales microarquitectónicas que reflejan cómo se distribuye el costo de ejecución entre el núcleo de cómputo y el subsistema de memoria. En este contexto, métricas como las instrucciones por ciclo (*Instructions Per Cycle*, IPC), la tasa de fallos de caché y los ciclos de estancamiento (*stall cycles*) resultan relevantes porque describen manifestaciones observables de la eficiencia de ejecución y de la presión sobre la jerarquía de memoria [9, 10].

El IPC expresa la cantidad promedio de instrucciones retiradas por ciclo de reloj y mide el grado en que el procesador transforma ciclos de ejecución en trabajo útil. Valores relativamente altos son consistentes con un flujo sostenido de actividad efectiva, mientras que valores bajos sugieren que una mayor fracción del tiempo de ejecución está siendo absorbida por latencias, dependencias o esperas por recursos [11, Ch. 6]. Por ello, el IPC puede enten-

derse como una señal del aprovechamiento efectivo del núcleo, aunque no basta por sí solo para caracterizar completamente el régimen de ejecución.

De forma complementaria, la tasa de fallos de caché describe la proporción de accesos que no pueden resolverse dentro de la jerarquía de caché y deben continuar hacia niveles de mayor latencia [11, Ch. 2]. En particular, la tasa de fallos en el último nivel de caché ofrece una señal relevante del peso relativo del acceso a memoria dentro del comportamiento global de la aplicación [11, Ch. 6]. A diferencia del número absoluto de fallos, una razón o porcentaje de fallos permite interpretar con mayor claridad la presión real sobre el subsistema de memoria.

En este mismo sentido, un *stall* puede definirse como un ciclo o conjunto de ciclos en los que el pipeline no logra avanzar al ritmo esperado porque el procesador debe esperar datos, recursos o la resolución de dependencias internas. Cuando estos estancamientos se asocian predominantemente a la latencia de memoria, indican que una mayor fracción del tiempo de ejecución está condicionada por la llegada de datos y no por la capacidad aritmética del núcleo [11, Ch. 6]. Resulta pertinente distinguir el origen del estancamiento: los que se producen en la etapa de captación y decodificación de instrucciones responden a causas distintas (fallos en la caché de instrucciones, predicción de saltos) de los que se producen en la etapa de ejecución, donde la espera por operandos procedentes de la jerarquía de memoria es una causa dominante. Esta segunda familia es la que guarda relación más directa con el régimen *memory-bound* y, por tanto, la de mayor valor discriminante para el problema abordado.

En conjunto, estas métricas no constituyen una formulación analítica exacta de los estados *compute-bound* y *memory-bound*, pero sí ofrecen una representación observable de aspectos complementarios del comportamiento de ejecución [9, 10]. El IPC refleja el aprovechamiento efectivo del núcleo, la tasa de fallos de caché la presión relativa sobre la jerarquía de memoria y los *stall cycles* las interrupciones en la continuidad del trabajo útil. Por ello, su valor teórico radica en que permiten describir patrones de ejecución consistentes con distintos regímenes de limitación del rendimiento [10].

Ventanas de muestreo y naturaleza temporal de la observación

Los contadores de hardware son acumulativos: su valor crece monótonamente desde el instante en que se habilitan. En consecuencia, una lectura individual no describe el comportamiento instantáneo de la aplicación, sino su historia agregada. La unidad de análisis adecuada no es la lectura, sino la diferencia entre dos lecturas consecutivas, que describe el trabajo realizado en el intervalo comprendido entre ambas. A este intervalo se le denomina ventana de muestreo.

La elección del período de muestreo establece un compromiso. Un período corto ofrece mayor resolución temporal y permite observar transiciones de fase más finas, pero incrementa la sobrecarga del propio instrumento y reduce el número de eventos acumulados en cada ventana, lo que degrada la relación señal–ruido de las métricas derivadas: una razón calculada sobre conteos muy pequeños es estadísticamente inestable, aun cuando el contador funcione correctamente. Un período largo produce métricas más estables pero promedia el comportamiento sobre intervalos que pueden abarcar varias fases distintas, difuminando precisamente la señal que se desea capturar.

Adicionalmente, el intervalo efectivo entre lecturas no coincide exactamente con el período nominal solicitado, dado que el hilo que realiza el muestreo está sujeto a la planificación de un sistema operativo de propósito general. Por esta razón, toda métrica derivada debe calcularse sobre el intervalo realmente transcurrido entre las dos lecturas, medido con un reloj monótono, y no sobre el período nominal configurado; la dispersión entre ambos constituye además un indicador de la calidad del muestreo.

1.1.4. Espacios de ejecución: *user-space* y *kernel-space*

Los sistemas operativos modernos organizan la ejecución del software en dominios de privilegio diferenciados, comúnmente denominados *user-space* y *kernel-space*. Esta separación se apoya en la arquitectura del procesador, que permite operar al menos en modo usuario y modo kernel. En modo usuario, el software solo puede acceder a memoria marcada para el espacio de usuario; cualquier intento de acceso a memoria del kernel produce una excepción de hardware. En modo kernel, en cambio, el procesador puede acceder tanto al espacio del kernel como al del usuario [12, ch. 2].

Esta distinción constituye un mecanismo fundamental de protección y control. Operaciones sensibles como la gestión de memoria, el acceso directo al hardware o las operaciones de entrada y salida requieren privilegios de kernel, lo que impide que los procesos ordinarios interfieran con estructuras críticas del sistema [12, ch. 2]. En este marco, *kernel-space* designa el dominio donde residen el núcleo del sistema operativo y los mecanismos que arbitran el acceso a los recursos físicos, mientras que *user-space* comprende programas, bibliotecas y entornos de ejecución que operan fuera de ese núcleo [13]. Para el software de aplicación, esto implica que el acceso a recursos privilegiados no ocurre de forma directa, sino mediante mecanismos de mediación del sistema operativo. En sistemas tipo Unix y Linux, esta mediación se realiza principalmente a través de las llamadas al sistema, que constituyen la interfaz entre los programas de *user-space* y los servicios del kernel [13]. Así, cuando una aplicación necesita abrir un archivo, reservar memoria o interactuar con dispositivos y contadores del

sistema, no actúa directamente sobre el hardware, sino que solicita al kernel la operación correspondiente dentro de un marco de control y aislamiento [12, ch. 2], [13].

Esta distinción resulta especialmente relevante en problemas de observabilidad y gestión del hardware. Dado que los programas en *user-space* no poseen acceso irrestricto a dispositivos ni a estructuras internas del sistema, cualquier mecanismo de monitoreo o control implementado fuera del kernel debe apoyarse en interfaces explícitamente expuestas por la plataforma. Por ello, la separación entre *user-space* y *kernel-space* no solo define un modelo de protección del sistema operativo, sino también el alcance y las restricciones de las soluciones software que buscan observar el comportamiento del hardware o influir sobre su estado operativo [12, ch. 2], [13].

1.1.5. Interfaces estándar de observabilidad e instrumentación

La observabilidad del comportamiento de sistemas heterogéneos CPU–GPU depende de interfaces que permitan acceder, desde software de alto nivel, a métricas internas de rendimiento, utilización y consumo energético sin requerir la modificación directa del kernel ni de los controladores del hardware. En este contexto, las plataformas modernas exponen mecanismos estándar que posibilitan la recolección de señales relevantes para el análisis del sistema. Entre ellas, destacan la interfaz `perf_event` en Linux para contadores de rendimiento del procesador, Intel RAPL para dominios energéticos de CPU, y la biblioteca NVIDIA Management Library (NVML) para telemetría operativa y energética de GPU NVIDIA [4, 14, 15, 16].

La interfaz `perf_event` y los modos de atribución

En sistemas Linux, `perf` constituye una de las interfaces estándar más utilizadas para acceder a la PMU del procesador. A través de la infraestructura `perf_event`, el kernel expone eventos de bajo nivel asociados a ciclos de reloj, instrucciones retiradas, fallos de caché y otros indicadores microarquitectónicos del comportamiento del núcleo [14]. En esencia, `perf` representa el puente entre los contadores de rendimiento hardware y su observación desde *user-space*, permitiendo instrumentar el comportamiento del procesador mediante una interfaz homogénea y ampliamente adoptada en Linux [14].

Un aspecto de esta interfaz resulta determinante para la construcción de un conjunto de datos válido: el alcance de atribución de la medida. Al abrir un contador, el solicitante especifica a qué se asocia el evento, y las alternativas no son equivalentes desde el punto de vista experimental. La medición de alcance de CPU contabiliza todo lo que ocurre en un

procesador determinado, incluyendo la actividad de procesos ajenos al experimento, por lo que la medida queda contaminada por el ruido del sistema. La medición asociada a un proceso concreto, en cambio, contabiliza únicamente la actividad de ese proceso, con independencia de sobre qué CPU sea planificado.

Este segundo modo requiere una precisión adicional cuando la carga medida es paralela. Un programa que crea hilos o procesos hijos distribuye su trabajo entre ellos, de modo que observar únicamente el proceso inicial subestimaría sistemáticamente la actividad real. Para cubrir este caso, la interfaz permite solicitar que el contador se herede por los descendientes creados a partir del momento en que se habilita, de forma que la medida abarque el árbol completo de ejecución de la aplicación medida y solo ese árbol. La combinación de atribución por proceso con herencia constituye, por tanto, el modo que ofrece una correspondencia más limpia entre lo que se mide y la carga que se pretende caracterizar, y es el adoptado en este trabajo.

Cabe señalar una restricción técnica asociada: cuando se solicita herencia, la interfaz no admite la lectura agrupada y consistente de varios contadores en una única operación. En consecuencia, cada evento debe leerse de forma independiente, lo que introduce un desfase temporal de orden microsegundo entre las lecturas de distintos eventos correspondientes a una misma ventana. Este desfase es despreciable frente a períodos de muestreo del orden del milisegundo, pero debe declararse como una característica conocida del instrumento.

Niveles de privilegio en el acceso a contadores

El acceso a la infraestructura de monitorización desde espacio de usuario no es irrestricto. Los sistemas Linux exponen un parámetro de configuración que establece el nivel de restricción aplicable a usuarios sin privilegios especiales, y cuyos valores habilitan progresivamente menos capacidades a medida que aumentan. En los niveles más restrictivos, un usuario ordinario puede medir los contadores asociados a sus propios procesos, pero no puede abrir eventos de alcance de sistema, categoría en la que se sitúan los contadores de *uncore* descritos en la sección 1.1.3.

Esta configuración tiene una consecuencia metodológica directa: la disponibilidad de la medición depende no solo del hardware, sino de la política de administración del sistema. Un mismo instrumento puede resultar plenamente funcional para la medición por proceso y, simultáneamente, incapaz de acceder a los contadores del controlador de memoria en la misma máquina, sin que ello constituya un defecto del instrumento. Por tanto, la caracterización de las capacidades efectivas de la plataforma de medición es un paso previo obligatorio al diseño de la campaña experimental, y las limitaciones detectadas deben trasladarse explícitamente

al alcance de las conclusiones.

Intel RAPL

De manera complementaria, Intel RAPL (*Running Average Power Limit*) proporciona una interfaz integrada en muchos procesadores Intel para observar el comportamiento energético del sistema a través de distintos dominios internos. Introducido originalmente como un mecanismo de modelado energético y posteriormente refinado en microarquitecturas más recientes, RAPL expone información de energía acumulada para dominios como el paquete del procesador, los núcleos, la memoria DRAM y, cuando existe, otros componentes específicos del sistema [15, 17]. Esta organización por dominios resulta conceptualmente relevante porque permite desagregar parcialmente el comportamiento energético del procesador y de la memoria principal, lo que facilita el análisis de la relación entre consumo energético y configuración operativa del hardware.

RAPL reporta energía mediante registros específicos del modelo, cuyos valores representan energía acumulada en unidades dependientes de la plataforma [17]. Estas lecturas no cubren de forma exhaustiva el consumo total del nodo y la disponibilidad de dominios depende de la microarquitectura y del modelo de CPU [17]. Además, al ser contadores acumulativos, su interpretación debe considerar la periodicidad de actualización y el posible desbordamiento de los registros [17]. Este último aspecto exige que el instrumento de medición registre el valor máximo del rango antes de iniciar la captura, de modo que un decremento aparente entre dos lecturas consecutivas pueda interpretarse correctamente como un desbordamiento y no como un consumo negativo. Aun así, RAPL es una de las interfaces más utilizadas en estudios de eficiencia energética y DVFS sobre CPU, porque ofrece una vía estandarizada y de baja intrusión para observar variaciones relativas de energía y potencia sin instrumentación externa especializada [4, 15, 17].

Debe notarse, además, que RAPL es una capacidad del procesador y no una interfaz universal: microarquitecturas anteriores a su introducción carecen por completo de ella, sin que exista alternativa por software. En consecuencia, la disponibilidad de RAPL constituye un criterio de admisibilidad para cualquier plataforma destinada a un estudio de eficiencia energética basado en medición interna.

NVIDIA Management Library

En el caso del acelerador gráfico, la interfaz estándar más extendida en plataformas NVIDIA es la NVIDIA Management Library (NVML). Esta biblioteca permite consultar información relacionada con el estado operativo de la GPU, incluyendo métricas de utilización,

potencia, memoria y otros parámetros relevantes del dispositivo [16]. A diferencia de `perf`, cuyo foco principal es la observación de eventos microarquitectónicos del procesador, NVML se orienta a la telemetría y gestión del acelerador como dispositivo completo, ofreciendo una vista del comportamiento de la GPU útil tanto para monitoreo como para análisis energético [4, 16].

Esta diferencia de granularidad es conceptualmente importante para el problema abordado. Las métricas que NVML expone describen el estado del dispositivo, no la intensidad operacional de un kernel concreto: la utilización reportada indica qué fracción del tiempo hubo trabajo activo, pero no cuántas operaciones por byte realizó ese trabajo. En consecuencia, la caracterización del régimen de ejecución en el dominio GPU no puede sustentarse únicamente en telemetría de gestión, y requiere una estrategia de medición distinta a la empleada en CPU.

En conjunto, `perf`, RAPL y NVML no cumplen funciones idénticas, pero sí conforman una base complementaria de observabilidad para sistemas heterogéneos. `perf` permite acceder a señales del comportamiento microarquitectónico del procesador; RAPL aporta una vía de observación energética de la CPU; y NVML extiende esta capacidad al dominio de la GPU [4, 14, 15, 16].

1.1.6. Gestión dinámica de frecuencia en sistemas operativos: DVFS, P-states y *governors*

La gestión dinámica de voltaje y frecuencia constituye uno de los mecanismos centrales de control energético en procesadores modernos. Su propósito general es ajustar el punto operativo del hardware mediante cambios coordinados en la frecuencia de reloj y el voltaje de alimentación, con el fin de reducir consumo y disipación térmica cuando la carga de trabajo no requiere el máximo desempeño disponible. En arquitecturas contemporáneas, este mecanismo no depende únicamente de una relación física entre voltaje y frecuencia, sino también de recursos hardware dedicados a la gestión de potencia, entre ellos reguladores de voltaje y unidades de control de potencia, que supervisan dominios funcionales del procesador y ejecutan solicitudes de cambio de estado operativo [18].

Desde la dimensión del sistema operativo, DVFS se expresa a través de abstracciones de estado que desacoplan la política software de los detalles eléctricos internos del procesador. En particular, el estándar ACPI define los *performance states* o P-states, que representan distintos niveles operativos de frecuencia y voltaje dentro del estado activo del procesador, y los C-states, que representan niveles progresivos de inactividad o reposo. En este esquema,

el estado P0 corresponde al mayor nivel de rendimiento disponible, mientras que estados superiores en numeración representan frecuencias progresivamente menores; de forma análoga, los C-states más profundos permiten mayores ahorros de energía a costa de mayores latencias de reactivación. Así, los P-states permiten modular el rendimiento de componentes activos, mientras que los C-states gestionan la inactividad de componentes o dominios no utilizados [18, 11].

Sobre esta base operan los *governors*, es decir, políticas que deciden cuándo y cómo cambiar el estado operativo del procesador. Un *governor* no implementa el mecanismo físico de DVFS, sino la lógica de control que interpreta señales del sistema y solicita transiciones entre P-states. En términos generales, estas políticas pueden agruparse en gobernadores de frecuencia fija y gobernadores dinámicos [18]. Las políticas de frecuencia fija sitúan al procesador en extremos operativos: **performance** mantiene de forma estática la frecuencia máxima, lo que favorece cargas sensibles a latencia, pero puede resultar energéticamente ineficiente en escenarios de baja utilización; en el extremo opuesto, **powersave** fija la frecuencia mínima, reduciendo la potencia instantánea a costa de un mayor tiempo de ejecución, por lo que este menor consumo no siempre implica menor energía total [18].

Para evitar estas dos situaciones extremas, Linux incorpora gobernadores dinámicos. **ondemand** ajusta la frecuencia según la carga media del procesador, elevándola cuando la utilización supera un umbral y reduciéndola cuando la carga disminuye. **conservative** sigue una lógica similar, pero realiza cambios más graduales. Por su parte, **schedutil** integra la decisión de frecuencia con el planificador del sistema operativo [18].

En Linux, esta política no actúa de forma aislada, sino sobre una jerarquía de componentes que va desde el hardware y sus registros de control hasta los controladores del kernel y la interfaz expuesta al espacio de usuario. El trabajo de Hebbar y Milenković [18] resume esta jerarquía mostrando que el control de potencia involucra el hardware de gestión energética, la BIOS, los controladores de transición de estado y, finalmente, los *governors* accesibles desde la capa de usuario. Dentro de este esquema, distinguen dos controladores principales: **acpi-cpufreq**, desarrollado por la comunidad de Linux y utilizado de forma genérica en múltiples plataformas, e **intel_pstate**, empleado como controlador por defecto en muchas generaciones de procesadores Intel. Ambos actúan como intermediarios entre la lógica de control del sistema operativo y la infraestructura interna de gestión de potencia del procesador.

La diferencia entre ambos controladores no es menor y tiene consecuencias operativas directas sobre cómo debe implementarse el control de frecuencia. Bajo **acpi-cpufreq**, Linux

expone una familia de *governors* genéricos, entre ellos `userspace`, que permite fijar explícitamente una frecuencia objetivo. En cambio, bajo `intel_pstate` el modelo cambia: solo se soportan `performance` y `powersave`, y no existe un *governor userspace*. En esta configuración, fijar un punto operativo concreto no se realiza escribiendo una frecuencia objetivo, sino restringiendo el rango permitido mediante los límites inferior y superior, de modo que el hardware quede confinado al valor deseado. Adicionalmente, `intel_pstate` puede aprovechar los denominados *hardware P-states*, en los cuales el procesador selecciona autónomamente los P-states a partir de la utilización observada, con menor intervención directa del sistema operativo [18]. Este punto es conceptualmente relevante porque muestra que la gestión de frecuencia en sistemas modernos no sigue un único modelo universal, sino que depende del grado de autonomía transferido al hardware y del acoplamiento entre firmware, kernel y microarquitectura; en consecuencia, un agente de control portable debe detectar el controlador activo y adaptar su estrategia de actuación en consecuencia.

Otro componente fundamental de este problema es la latencia de conmutación de frecuencia. Las transiciones entre P-states no son instantáneas: implican un costo temporal asociado al procesamiento de la solicitud y al ajuste efectivo de frecuencia y voltaje. Según lo reportado en [18], estas transiciones pueden tomar aproximadamente 10 ms, mientras que con *hardware P-states* pueden reducirse a alrededor de 1 ms. Aunque estos valores dependen de la plataforma, la implicación conceptual es general: cualquier política de control que pretenda modificar frecuencias de manera continua debe considerar que el costo de transición puede volverse relevante si las decisiones se toman con excesiva frecuencia o si las fases de ejecución cambian más rápido que la capacidad efectiva del sistema para materializar el nuevo estado.

Dominios de frecuencia y aislamiento experimental

Una propiedad de la plataforma que condiciona tanto la validez experimental como la seguridad de la actuación es la granularidad del dominio de frecuencia, es decir, el conjunto de procesadores lógicos cuya frecuencia se ve afectada simultáneamente por una misma solicitud de cambio. En algunas generaciones de hardware, este dominio abarca un zócalo completo, de modo que fijar la frecuencia de un núcleo modifica también la de todos los demás núcleos del zócalo; en otras, el dominio se reduce a cada núcleo individual.

Esta distinción es relevante en dos sentidos. Desde la perspectiva de la validez interna, si el dominio de frecuencia excede el conjunto de núcleos asignados al experimento, la actuación puede alterar las condiciones de ejecución de procesos ajenos, y recíprocamente, la actuación de terceros puede alterar las del experimento. Desde la perspectiva operativa en infraestructuras compartidas, escribir la frecuencia de un dominio que excede la asignación

propia constituye una interferencia con otros usuarios del sistema. Por ello, la determinación del dominio real de frecuencia — consultada en la plataforma y no supuesta a partir del modelo de procesador — constituye una verificación previa obligatoria antes de habilitar cualquier actuación sobre la frecuencia.

De forma relacionada, la presencia de *simultaneous multithreading* implica que dos procesadores lógicos pueden compartir los recursos de ejecución de un mismo núcleo físico. Cuando el experimento no controla explícitamente esta relación, dos hilos del mismo experimento — o el hilo de medición y el hilo medido — pueden competir por los mismos recursos físicos, alterando tanto el rendimiento observado como la atribución de eventos. En consecuencia, la política de asignación de procesadores lógicos debe declararse explícitamente y verificarse contra la topología real de la máquina.

Particularidades de DVFS en GPU

En GPU, la gestión dinámica de voltaje y frecuencia no puede interpretarse exactamente bajo el mismo esquema conceptual que en CPU. Mientras en procesadores convencionales la discusión sobre DVFS suele centrarse en estados de rendimiento asociados al procesador y a sus núcleos, en aceleradores gráficos modernos el comportamiento energético y temporal de una aplicación depende de la interacción entre múltiples dominios funcionales, en particular el dominio de cómputo y el dominio de memoria. De forma general, las arquitecturas GPU modernas distinguen al menos un dominio asociado al procesamiento, que agrupa los multiprocesadores de *streaming* y parte de la jerarquía interna de caché, y un dominio asociado a la memoria principal del dispositivo. Esta separación implica que cambios en la frecuencia del núcleo y cambios en la frecuencia de memoria no producen necesariamente el mismo efecto sobre el tiempo de ejecución ni sobre la potencia consumida. Por ello, la respuesta de una aplicación a DVFS en GPU no depende únicamente de la capacidad de cómputo disponible, sino también de la presión ejercida sobre el subsistema de memoria y de la relación entre ambos dominios [19].

La distinción entre cargas *compute-bound* y *memory-bound* es especialmente importante en GPU. Cuando el rendimiento está limitado por el cómputo, la frecuencia del dominio de procesamiento impacta más directamente el tiempo de ejecución. En cambio, cuando predomina la limitación por memoria, la sensibilidad del rendimiento frente a la frecuencia del núcleo disminuye y el dominio de memoria pasa a ser más determinante. Por ello, el efecto de DVFS en GPU debe analizarse según la relación entre cómputo y memoria, y no solo como una variación de la frecuencia del núcleo [19, 5]. Además, en GPU la potencia, el rendimiento y la energía no varían de forma independiente con la frecuencia. Reducir

la frecuencia puede bajar la potencia instantánea, pero no necesariamente la energía total si el tiempo de ejecución aumenta demasiado; de forma análoga, subir la frecuencia puede mejorar el rendimiento sin traducirse en una mejora proporcional de la eficiencia energética. Por tanto, el análisis de DVFS debe considerar conjuntamente tiempo de ejecución y consumo de potencia, especialmente cuando existen dominios de frecuencia diferenciados [19, 5].

En síntesis, la especificidad de DVFS en GPU radica en que el problema de control no recae sobre un único dominio homogéneo, sino sobre una arquitectura donde coexisten subsistemas con sensibilidades distintas frente a la frecuencia. Esta característica diferencia conceptualmente la gestión de frecuencia en GPU respecto de la CPU y justifica que su análisis teórico incorpore explícitamente la relación entre cómputo, memoria y consumo energético [19].

1.1.7. Cargas de trabajo: kernels, benchmarks y microbenchmarks

En computación de alto rendimiento, el análisis del comportamiento de una aplicación rara vez se realiza únicamente a nivel del programa completo. En su lugar, resulta habitual estudiar unidades de ejecución más acotadas, patrones de carga bien definidos y conjuntos estandarizados de pruebas que permitan observar de manera controlada el rendimiento, el uso de recursos y la sensibilidad del sistema frente a distintos mecanismos de gestión.

Kernel como unidad de ejecución

En el ámbito de HPC, el término kernel no se refiere al núcleo del sistema operativo, sino a una rutina computacional bien delimitada que concentra una fracción relevante del trabajo de una aplicación. Un kernel suele representar una operación dominante dentro del flujo de ejecución, como una multiplicación de matrices, una transformación rápida de Fourier, una actualización de *stencil* o un recorrido estructurado sobre memoria. Debido a que estas rutinas encapsulan patrones recurrentes de acceso a datos y de uso de unidades aritméticas, su análisis permite estudiar con mayor precisión el comportamiento de una aplicación que si se considera únicamente el programa completo [20].

Bajo un enfoque de rendimiento, el kernel constituye una unidad útil de observación porque suele exhibir una firma computacional más estable que la aplicación global. Mientras una aplicación completa puede combinar distintas fases con demandas heterogéneas sobre cómputo, memoria y comunicación, un kernel tiende a concentrar un patrón dominante de ejecución. Por ello, el estudio de kernels resulta especialmente pertinente para analizar regímenes *compute-bound* y *memory-bound*, ya que permite aislar con mayor claridad la relación

entre intensidad computacional, uso de memoria y sensibilidad a cambios en la frecuencia de operación.

Benchmark y microbenchmark

Un benchmark es una carga de trabajo diseñada o seleccionada con el propósito de medir y comparar el comportamiento de un sistema bajo condiciones reproducibles. En el contexto de HPC, los benchmarks pueden representar aplicaciones completas, miniaplicaciones o kernels representativos de dominios científicos concretos. Su valor radica en que permiten evaluar rendimiento, consumo energético o escalabilidad sobre una base común, facilitando la comparación entre arquitecturas, configuraciones o políticas de control [11, Ch. 12].

Dentro de esta categoría, un microbenchmark constituye una forma más reducida y controlada de benchmark, orientada a aislar el comportamiento de un componente, mecanismo o patrón específico. Un microbenchmark no pretende reproducir toda la complejidad de una aplicación real, sino capturar de manera precisa una propiedad particular del sistema, por ejemplo, el rendimiento de una operación aritmética intensiva, el ancho de banda de memoria o la latencia asociada a un acceso repetitivo [11, Ch. 12]. En consecuencia, mientras los benchmarks ofrecen una visión más cercana al comportamiento de cargas reales, los microbenchmarks permiten estudiar fenómenos de bajo nivel con menor interferencia de factores externos. Esta distinción es funcionalmente relevante para el presente trabajo, dado que los microbenchmarks de ancho de banda y de densidad aritmética son los instrumentos adecuados para la calibración empírica de los techos del modelo Roofline descrita en la sección 1.1.2, mientras que los benchmarks representativos de aplicaciones científicas constituyen el objeto de caracterización.

Verificación de la unidad de trabajo reportada

Las suites de benchmarks científicos suelen reportar, al finalizar su ejecución, una medida agregada de la tasa de trabajo realizada. Sin embargo, esa medida no siempre corresponde a operaciones de punto flotante: algunos programas de una misma suite cuantifican su trabajo en unidades distintas, como números aleatorios generados o claves ordenadas, según la naturaleza del problema que resuelven. Dado que el numerador de la intensidad operacional (ecuación (1.4)) debe expresarse en operaciones de punto flotante para ser comparable con un techo P_{pico} calibrado en esas mismas unidades, emplear indistintamente la tasa reportada por cualquier programa introduciría un error de unidades que invalidaría la clasificación resultante. Por consiguiente, la verificación explícita de la unidad en que cada carga reporta su trabajo constituye un criterio de admisión al catálogo experimental, y no un detalle de

implementación.

1.1.8. Aprendizaje automático supervisado ligero

El aprendizaje supervisado es un paradigma del aprendizaje automático cuyo objetivo es inferir una función que mapea un conjunto de variables de entrada hacia una variable de salida a partir de datos de entrenamiento previamente etiquetados. En este enfoque, el modelo aprende regularidades a partir de ejemplos históricos en los que la clase correcta ya es conocida, con el propósito de generalizar ese conocimiento sobre observaciones no vistas. De manera general, el conjunto de datos se divide en subconjuntos de entrenamiento y prueba, de modo que el primero se utiliza para ajustar el modelo y el segundo para examinar su capacidad de generalización fuera de los ejemplos usados durante el aprendizaje [21].

Tipos de modelos ligeros de clasificación supervisada

En contextos donde la decisión debe ejecutarse con restricciones de tiempo y recursos, resulta especialmente relevante la noción de modelo ligero. En términos generales, un clasificador ligero es aquel cuya inferencia puede realizarse con baja complejidad computacional y consumo acotado de memoria, manteniendo al mismo tiempo una capacidad razonable de generalización. Esta ligereza no depende solo del nombre del algoritmo, sino también de la complejidad efectiva del modelo entrenado y del costo aceptable de predicción en el entorno donde será utilizado. Entre los clasificadores supervisados de uso más extendido se encuentran los árboles de decisión, los bosques aleatorios, la regresión logística, las máquinas de soporte vectorial y los métodos basados en vecinos cercanos [21]. Cada uno de ellos impone un criterio distinto de separación entre clases y presenta compromisos diferentes entre interpretabilidad, flexibilidad, robustez y costo de inferencia.

Bosques Aleatorios (*Random Forest*). El algoritmo de Bosques Aleatorios es un método de aprendizaje automático supervisado fundamentado en el paradigma del aprendizaje en conjunto. Este enfoque se basa en el principio de que la combinación de múltiples modelos predictivos base puede resolver problemas computacionales complejos para generar una predicción final más robusta y precisa que la de un modelo individual [22]. En esta arquitectura, los modelos base utilizados son los árboles de decisión. Durante la fase de entrenamiento, el algoritmo construye un bosque compuesto por una multitud de estos árboles de forma independiente. Cuando el modelo se enfrenta a datos no observados, cada árbol individual emite su propia predicción y el algoritmo agrega estos resultados para tomar una decisión final; en tareas de clasificación, la salida es la clase con la votación mayoritaria [22].

Para garantizar que los árboles no sean idénticos y aporten diversidad al conjunto, el modelo implementa dos mecanismos estocásticos. El primero es la agregación de *bootstrap*, mediante el cual cada árbol se entrena utilizando una muestra aleatoria diferente extraída del conjunto de entrenamiento original con reemplazo [22]. El segundo es la aleatoriedad de características, que establece que, al construir la división en cada nodo del árbol, no se evalúan todas las variables disponibles, sino un subconjunto aleatorio de estas [22]. La combinación de ambos mecanismos garantiza una baja correlación entre los árboles individuales, lo que mitiga directamente el problema de sobreajuste que suelen sufrir los árboles de decisión profundos [22].

Regresión Logística. La regresión logística es un algoritmo de clasificación supervisada utilizado para predecir una salida categórica. A diferencia de la regresión lineal, que modela variables continuas, la regresión logística estima la probabilidad de pertenencia a una clase, por lo que su salida queda acotada en el intervalo $[0, 1]$. En su forma más común, la regresión logística binaria modela eventos dicotómicos [23]. Su formulación parte de un predictor lineal de la forma:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n \quad (1.7)$$

Donde $x_1, x_2, \dots, x_n$ son las variables de entrada y $\beta_0, \beta_1, \dots, \beta_n$ los coeficientes del modelo. Como este predictor lineal puede tomar valores en $(-\infty, \infty)$, no puede interpretarse directamente como probabilidad [23]. Para resolverlo, la regresión logística aplica una transformación sigmoide, que restringe la salida al intervalo $[0, 1]$:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-z}} \quad (1.8)$$

De este modo, la salida del modelo puede interpretarse como la probabilidad de que una observación pertenezca a la clase positiva. Conceptualmente, sus principales ventajas son la simplicidad, el bajo costo de inferencia y la facilidad de interpretación, mientras que su principal limitación es que induce una frontera de decisión lineal, por lo que puede resultar insuficiente cuando la separación entre clases depende de relaciones fuertemente no lineales [23].

XGBoost. XGBoost (*eXtreme Gradient Boosting*) es un algoritmo supervisado de clasificación y regresión basado en ensambles de árboles de decisión potenciados secuencialmente. Su principio general consiste en construir múltiples árboles débiles de manera aditiva, de

modo que cada nuevo árbol aprenda a corregir los errores cometidos por los anteriores. A diferencia de un árbol de decisión individual, que puede presentar alta varianza y tendencia al sobreajuste, XGBoost incrementa la robustez predictiva al combinar secuencialmente múltiples árboles. Cada iteración parte de los residuos o errores del modelo actual y entrena un nuevo árbol que aproxime la corrección necesaria, lo que le permite capturar relaciones no lineales e interacciones complejas entre variables de entrada [24].

Una característica distintiva de XGBoost frente a implementaciones más simples de *gradient boosting* es que incorpora regularización directamente en el objetivo de aprendizaje, lo que ayuda a controlar la complejidad del modelo y a mitigar el sobreajuste. Además, la biblioteca está diseñada para ofrecer alta eficiencia computacional, con soporte para procesamiento paralelo y distribuido, mecanismos de optimización orientados a caché y tratamiento explícito de valores faltantes [24]. Atendiendo a su naturaleza funcional, XGBoost puede entenderse como un modelo supervisado de clasificación más expresivo que la regresión logística y más robusto que un árbol individual, aunque a costa de una mayor complejidad estructural.

1.1.9. Producto Energía–Retardo (EDP)

La evaluación de estrategias de optimización energética en sistemas de alto rendimiento no puede basarse únicamente en minimizar la energía consumida o el tiempo de ejecución por separado. Una política que reduzca el consumo energético a costa de una degradación severa del rendimiento puede resultar globalmente inconveniente; de forma análoga, una configuración orientada exclusivamente al máximo desempeño puede implicar un costo energético desproporcionado. En este contexto, el Producto Energía–Retardo se emplea como una métrica fusionada que integra ambas dimensiones en una sola magnitud [25]. De forma general, esta familia de métricas puede expresarse como:

$$EDP = E \cdot T^w \quad (1.9)$$

En donde E representa la energía consumida durante la ejecución, T el tiempo de ejecución, y w un factor de ponderación que permite ajustar la importancia relativa del retardo dentro de la métrica [25]. Cuando $w = 1$ se obtiene el EDP clásico; cuando $w = 2$, se obtiene una variante que penaliza más fuertemente la degradación del tiempo de ejecución, comúnmente asociada al *Energy–Delay² Product* [5, 25].

La utilidad del EDP radica en que evita decisiones sesgadas por una sola dimensión de análisis. En [25] se remarca que se trata de una métrica construida para observar simultánea-

mente múltiples criterios en una sola función, mientras que en [5] se emplea como mecanismo para seleccionar configuraciones DVFS que logren un equilibrio entre ahorro energético y degradación del desempeño. En esa misma línea, se advierte que reducir únicamente la potencia puede inducir pérdidas de rendimiento que terminen aumentando la energía consumida, razón por la cual una función multiobjetivo basada en EDP resulta más adecuada para este tipo de problemas [5].

1.2. Marco Legal

1.3. Estado del Arte

La gestión energética en sistemas de alto rendimiento ha evolucionado desde enfoques estáticos de reducción de potencia hacia estrategias cada vez más adaptativas, orientadas a preservar el rendimiento sin incurrir en costos energéticos desproporcionados. En este contexto, el DVFS y las técnicas afines de *frequency capping* y *power capping* se han consolidado como mecanismos de control relevantes tanto en procesadores como en aceleradores GPU. Sin embargo, la eficacia de estos mecanismos depende de forma crítica del comportamiento dinámico de la carga de trabajo, lo que ha impulsado el desarrollo de modelos de caracterización, predicción y control cada vez más sofisticados [1, 2, 4, 5, 19].

Una primera línea de trabajo corresponde a la evaluación empírica del impacto de DVFS sobre aplicaciones y kernels HPC. Calore et al. analizaron técnicas DVFS sobre procesadores y aceleradores modernos desde el punto de vista del usuario, mostrando que el beneficio energético no es uniforme y que depende del carácter *compute-bound* o *memory-bound* de las partes críticas de la aplicación. Este resultado es relevante porque confirma que la optimización energética no puede formularse como una política global única, sino como una decisión sensible al perfil de ejecución de cada kernel o fase [4]. En una línea similar, Simsek et al. estudiaron simulaciones astrofísicas sobre GPU y mostraron que la instrumentación del código y el ajuste estático y dinámico de frecuencia permiten reducir el consumo energético con pérdidas moderadas de rendimiento, reportando reducciones de energía por GPU de hasta 7.82 % con una pérdida de rendimiento de 2.95 % [1]. Más recientemente, Costa et al. evaluaron *frequency capping* y *power capping* en un sistema exascale, observando que la efectividad relativa de cada mecanismo depende tanto de la naturaleza de la aplicación como de la escala de ejecución; en particular, *frequency capping* tendió a ofrecer mejores resultados energéticos a escala, mientras *power capping* resultó más adecuado en ciertos escenarios de nodo único o utilización irregular [2].

Una segunda línea de investigación se centra en los métodos *offline* de selección de frecuencia óptima. En este grupo destaca el trabajo de Guerreiro et al., quienes propusieron un esquema de clasificación de aplicaciones GPGPU consciente de DVFS, capaz de inferir, a partir de eventos de hardware recolectados a una sola frecuencia, cómo cambiarán tiempo de ejecución, potencia y energía al recorrer el resto del espacio de frecuencias. Su propuesta permitió encontrar pares de frecuencias casi óptimos, con ahorros medios de energía de 16 % a 20 % y picos de hasta 36 %, según la arquitectura evaluada [19]. De forma complementaria, Ali et al. desarrollaron un método automatizado y portable para seleccionar la frecuencia óptima de GPU a partir de tres etapas: caracterización de características, modelado analítico de potencia y tiempo de ejecución, y selección multiobjetivo mediante EDP y ED²P. Su metodología requiere recolectar métricas en una configuración base y luego estimar el comportamiento en el resto del espacio DVFS, alcanzando ahorros promedio de 29.6 % de energía con 5.2 % de pérdida de rendimiento en una arquitectura y 22.6 % con 4.7 % en otra [5]. Estos trabajos constituyen antecedentes sólidos porque muestran que el espacio DVFS puede explorarse sin fuerza bruta y con buena portabilidad. No obstante, comparten una limitación importante: su lógica de decisión es esencialmente *offline* o precomputada, por lo que no aborda de manera explícita la adaptación fina a la multifasicidad intra-ejecución de aplicaciones HPC generales.

Una tercera línea aborda la caracterización y clasificación de cargas o kernels a partir de telemetría de bajo nivel. Shekofteh et al. demostraron que la clasificación de kernels GPU puede realizarse con un conjunto reducido de métricas, seleccionadas para minimizar *overhead* sin sacrificar precisión, lo que confirma que la identificación *online* de comportamientos como *compute-bound* y *memory-bound* es técnicamente viable. Sin embargo, su objetivo principal fue mejorar la planificación y co-ejecución de kernels, no gobernar DVFS [9]. De manera análoga, Littman y Deakin exploraron el uso de aprendizaje automático para clasificar límites de rendimiento a partir de contadores de rendimiento, apoyando la idea de que la frontera *compute-bound/bandwidth-bound* puede inferirse desde datos y no solo mediante inspección manual del modelo Roofline [10]. En conjunto, estos trabajos respaldan la hipótesis de que una fase de ejecución puede representarse mediante una firma microarquitectónica observable, aunque no proponen por sí mismos un lazo de control energético que traduzca dicha clasificación en decisiones DVFS.

La cuarta línea corresponde a los mecanismos *online* de control dinámico durante ejecución. El antecedente más fuerte en esta categoría es DRLCAP, que propone un marco general de *frequency capping* de GPU en tiempo de ejecución basado en aprendizaje por refuerzo profundo. DRLCAP monitoriza información a nivel de sistema para detectar cambios

de fase, aprende una política adaptativa y reduce en promedio 22 % de la energía de GPU con menos de 3 % de degradación en arquitecturas NVIDIA, además de reportar resultados sobre AMD [26]. Este trabajo demuestra que el control *online* de frecuencia de GPU es factible y efectivo, pero también deja clara una diferencia conceptual frente a enfoques más ligeros: se trata de una política basada en aprendizaje por refuerzo profundo, orientada a GPU, sin formular explícitamente el problema como clasificación supervisada ligera de fases seguida de una política discreta interpretable. Por tanto, aunque constituye un antecedente directo, no agota el espacio de soluciones posibles para agentes ligeros en espacio de usuario.

Finalmente, una restricción frecuentemente subestimada en la literatura es el *overhead* del propio mecanismo de control. Velicka et al. mostraron que la latencia de conmutación de frecuencia en GPU no es despreciable, varía entre arquitecturas y entre pares de frecuencias, y puede llegar a ser suficientemente alta como para comprometer el beneficio de políticas excesivamente reactivas [27]. Esta observación es central, porque implica que un esquema *online* no debe limitarse a detectar fases correctamente: también debe amortizar el costo del cambio de frecuencia y evitar decisiones demasiado finas o frecuentes.

Aun cuando la literatura reciente ha avanzado en control dinámico de frecuencia durante ejecución, la mayoría de los trabajos revisados se concentra en el dominio GPU de forma aislada. DRLCAP, por ejemplo, propone un marco *online* guiado por aprendizaje por refuerzo profundo, pero su problema de control está centrado en la GPU y no en una política coordinada sobre CPU y GPU dentro de un nodo heterogéneo [26]. Del mismo modo, los métodos de selección óptima de frecuencia de Ali et al. y los modelos de clasificación conscientes de DVFS de Guerreiro et al. se enfocan en la selección de configuraciones para GPU [5, 19]. Aunque existen propuestas de coordinación entre CPU, GPU y memoria, como SparseDVFS, estas se desarrollan en el contexto de inferencia de redes neuronales en dispositivos *edge*, con una lógica de partición por bloques y una señal de control centrada en la esparsidad de operadores, por lo que su alcance, supuestos de carga y entorno de despliegue difieren sustancialmente del problema de HPC científico general abordado aquí [28].

En conjunto, esta revisión muestra que la literatura ya dispone de componentes importantes del problema, aunque todavía de forma fragmentada: existen métodos *offline* robustos para seleccionar frecuencias óptimas, mecanismos de caracterización y clasificación de cargas, y controladores *online* avanzados para GPU. Sin embargo, sigue abierta una brecha en torno a soluciones ligeras, implementables en espacio de usuario, que integren caracterización *online* de fases de ejecución, telemetría estándar de baja intrusión y decisiones DVFS coordinadas sobre CPU y GPU en nodos heterogéneos de alto rendimiento. En ese sentido, el aporte de este trabajo no radica en afirmar que la clasificación de fases o el control *online* sean

completamente inéditos, sino en proponer una integración distinta: un agente ligero, implementable sin modificaciones al kernel, que utilice modelos supervisados clásicos para inferir el régimen de ejecución y traduzca esa inferencia en decisiones DVFS de bajo *overhead*, con foco explícito en la optimización del Producto Energía–Retardo.

Capítulo 2

Metodología

2.1. Enfoque metodológico

La presente investigación se desarrolla bajo un enfoque cuantitativo de tipo experimental aplicado, orientado al cumplimiento del objetivo general del proyecto. El propósito metodológico consiste en determinar en qué medida un agente de gestión DVFS permite maximizar la eficiencia energética sin degradar significativamente el rendimiento de la aplicación en ejecución. Dicho impacto se evalúa a través de métricas cuantificables y reproducibles, principalmente el tiempo de ejecución, la energía consumida y el Producto Energía–Retardo (EDP).

El carácter experimental se sustenta en la manipulación deliberada de una variable independiente — el estado de frecuencia impuesto al procesador — sobre un conjunto controlado de cargas de trabajo, observando su efecto sobre variables dependientes de rendimiento y consumo. El carácter aplicado responde a que el producto de la investigación no es únicamente conocimiento descriptivo sobre el comportamiento energético de las cargas, sino un artefacto software funcional cuya utilidad se evalúa empíricamente frente a las alternativas ya disponibles en el sistema operativo.

El desarrollo se estructura en cuatro fases secuenciales e interdependientes, que abarcan desde la caracterización de cargas y extracción de telemetría hasta la validación empírica del sistema. Este capítulo describe en detalle el diseño de la primera fase, por ser la que se encuentra ejecutada al momento de redacción del presente documento, y presenta el diseño previsto para las fases restantes.

2.2. Diseño metodológico

2.2.1. Población y muestra

La población de estudio está constituida por el conjunto de aplicaciones ejecutables en sistemas heterogéneos CPU–GPU. Debido a la imposibilidad de abarcar la totalidad de estas, se define una muestra no probabilística de tipo intencional, compuesta por benchmarks y microbenchmarks representativos de regímenes de ejecución contrastantes. Esta selección permite inducir comportamientos diferenciados y obtener evidencia experimental suficiente para construir un conjunto de datos útil para el entrenamiento del clasificador. El uso de benchmarks representativos y de una muestra intencional es coherente con la necesidad de trabajar con cargas controladas y reproducibles dentro del alcance de un proyecto de pregrado.

El criterio de inclusión en el catálogo experimental combina tres condiciones. En primer lugar, la carga debe disponer de una verificación interna de corrección que permita descartar ejecuciones inválidas, de modo que una corrida cuyo resultado numérico no sea correcto no contamine el conjunto de datos. En segundo lugar, la carga debe reportar de forma explícita el volumen de trabajo realizado, requisito necesario para estimar el numerador de la intensidad operacional según lo descrito en la sección 1.1.2. En tercer lugar, y de acuerdo con lo argumentado en la sección 1.1.7, ese volumen de trabajo debe estar expresado en operaciones de punto flotante; las cargas que cuantifican su trabajo en otras unidades quedan excluidas del catálogo de caracterización, con independencia de su relevancia como benchmarks en otros contextos.

2.2.2. Plataforma experimental

Los experimentos se ejecutan sobre un nodo de cómputo heterogéneo cuyas características se resumen en la Tabla 2.1. La selección de la plataforma no fue arbitraria: además de la disponibilidad de un acelerador gráfico instrumentable mediante NVML, se estableció como condición de admisibilidad la presencia de una interfaz de medición energética interna accesible sin privilegios administrativos, dado que, como se argumentó en el marco conceptual, dicha capacidad es una propiedad del procesador que no admite sustitución por software.

La identidad precisa de la microarquitectura no es un dato meramente descriptivo en este trabajo: varios de los eventos crudos de PMU que el instrumento utiliza (sección 1.1.5) carecen de mapeo genérico en el kernel de este nodo y se abren mediante una codificación específica del procesador, verificada empíricamente y restringida en el código a coincidir exactamente

con el par `cpu family/model` de la Tabla 2.1; ese mismo gateo es lo que impide, por diseño, que el instrumento aplique sin verificación una codificación cruda a un nodo distinto de este.

Tabla 2.1: Características de la plataforma experimental.

Componente	Descripción
Procesador	$2 \times$ Intel Xeon Gold 5315Y
Microarquitectura	Ice Lake-SP (10 nm, generación de servidor lanzada en 2021)
Identificador CUID (<code>cpu family/model</code>)	6 / 106 — el mismo par leído en tiempo de ejecución desde /
Extensiones SIMD relevantes	AVX2, FMA3, AVX-512 (<code>avx512f</code> , <code>avx512bw</code> , <code>avx512cd</code> , <code>avx512dq</code>)
Núcleos	8 núcleos físicos por zócalo, 2 hilos por núcleo (32 lógicos)
Nodos NUMA	2 (uno por zócalo)
Jerarquía de caché	L1d 48 KiB, L1i 32 KiB, L2 1.25 MiB por núcleo; L3 12 MiB
Tamaño de línea de caché	64 bytes
Controlador de frecuencia	<code>intel_pstate</code>
Rango de frecuencia	0.8–3.6 GHz
Dominio de frecuencia	Por núcleo lógico
Medición energética	Intel RAPL (dominios de paquete y DRAM por zócalo)
Acelerador	NVIDIA A100 PCIe 40 GB
Gestor de recursos	Slurm

Las condiciones de ejecución se controlan mediante asignación exclusiva del nodo a través del gestor de recursos, de modo que ninguna carga ajena al experimento comparta el procesador durante las mediciones. Los procesadores lógicos empleados se restringen a hilos primarios de un mismo zócalo, evitando la asignación simultánea de hilos hermanos del mismo núcleo físico, de acuerdo con lo expuesto en la sección 1.1.6.

2.2.3. Instrumentos de recolección

La recolección de telemetría se implementa mediante un instrumento propio, desarrollado específicamente para este trabajo, compuesto por dos capas con responsabilidades separadas.

La capa inferior es un ejecutor escrito en C++ que lanza la carga de trabajo como proceso hijo y adjunta los contadores de hardware sobre ella. El diseño de esta capa responde directamente a los requisitos de atribución expuestos en la sección 1.1.5: el proceso hijo se detiene inmediatamente después de su creación y antes de ejecutar la primera instrucción de la carga, momento en el cual el proceso padre abre los contadores asociados a ese identificador de proceso con herencia habilitada; solo entonces se reanuda la ejecución. Este protocolo

garantiza que ninguna instrucción de la carga medida se ejecute fuera del intervalo de observación y que la medida abarque el árbol completo de procesos e hilos que la carga genere, sin incluir actividad ajena.

El muestreo se realiza mediante un hilo productor dedicado, fijado a un procesador lógico distinto de los asignados a la carga, que a intervalos regulares lee los contadores y deposita las muestras en una estructura circular de productor y consumidor únicos. Un hilo consumidor, igualmente fijado a un procesador lógico separado, extrae las muestras y las persiste. Esta separación evita que la actividad de escritura a disco interfiera con la periodicidad del muestreo, y el aislamiento de ambos hilos respecto de los núcleos medidos reduce su interferencia sobre la carga observada. La ausencia de pérdidas en la estructura circular se registra como indicador de integridad de cada corrida.

La capa superior es un orquestador escrito en Python, responsable de la lógica experimental: interpreta el manifiesto que define la campaña, detecta las capacidades de la plataforma, ejecuta las verificaciones previas, realiza la calibración, planifica y lanza cada combinación experimental, aplica el estado de frecuencia correspondiente, valida el resultado de cada corrida y transforma las muestras crudas en el conjunto de datos final. La separación entre ambas capas responde a un criterio de responsabilidad: la capa C++ resuelve la medición de baja latencia, donde el costo de la instrumentación es crítico, mientras que la capa Python resuelve la lógica experimental, donde prima la claridad y la trazabilidad sobre el rendimiento.

2.2.4. Variables observadas

Antes de describir las señales capturadas, conviene precisar qué se entiende por ventana de muestreo, término empleado de aquí en adelante. El hilo productor descrito en la sección anterior toma una lectura de los contadores cada cierto período configurado en la campaña; cada lectura individual es un valor acumulativo, y por sí sola no describe el comportamiento de un intervalo concreto. Una ventana de muestreo se define como el intervalo delimitado por dos lecturas consecutivas, y su contenido — el trabajo realizado durante ese intervalo — se obtiene restando la lectura anterior de la lectura actual, conforme a lo argumentado en la sección 1.1.3. Por tanto, una ventana no es un conjunto o agregado de varias muestras, sino el resultado de un único par de lecturas consecutivas; se requieren dos lecturas para producir una ventana, de modo que la primera lectura de cada corrida no genera ventana alguna, al carecer de una lectura previa contra la cual diferenciarse (condición registrada en la Tabla 2.5 como “sin diferencia previa”). La duración real de una ventana corresponde al tiempo efectivamente transcurrido entre ambas lecturas, medido con un reloj monótono, y no al período nominal configurado.

La Tabla 2.2 resume las señales de telemetría capturadas en cada ventana de muestreo. El conjunto de eventos se dimensionó para permanecer dentro del presupuesto de contadores físicos simultáneos de la plataforma, verificado empíricamente sobre la máquina en lugar de asumirse a partir del modelo de procesador, con el fin de evitar el error de extrapolación asociado a la multiplexación descrito en la sección 1.1.3.

Tabla 2.2: Señales de telemetría capturadas por ventana de muestreo.

Señal	Propósito
Instrucciones retiradas	Base para IPC, tasa de instrucciones y respaldo por prorrateo de FLOPs cuando la medición directa no está disponible.
Ciclos de reloj	Base para IPC y para la fracción de ciclos de estancamiento.
Referencias a caché	Denominador de la tasa de fallos.
Fallos de caché	Estimación de bytes movidos y de la presión sobre memoria.
Ciclos de estancamiento en ejecución	Señal directa de espera por operandos, asociada al régimen <i>memory-bound</i> .
Líneas transferidas a L2	Señal complementaria de tráfico de memoria, empleada como contraste independiente de la estimación basada en fallos de caché.
Operaciones de punto flotante retiradas (escalar, 128B, 256B, 512B)	Medición directa de FLOPs por ventana, ponderada por ancho de registro vectorial (ecuación (1.5)); fuente principal del numerador de la intensidad operacional.
Tiempo habilitado y tiempo contando	Diagnóstico de multiplexación y escalado de la medida.
Energía acumulada por dominio RAPL	Cálculo de energía y potencia por ventana.

A partir de estas señales se derivan, para cada ventana, las métricas de eficiencia descritas en el marco conceptual — instrucciones por ciclo, tasa de fallos del último nivel de caché, fallos por cada mil instrucciones, fracción de ciclos de estancamiento — así como la intensidad operacional y las magnitudes energéticas. Todas las tasas se calculan sobre el intervalo realmente transcurrido entre lecturas consecutivas, conforme a lo argumentado en la sección

1.1.3.

2.3. Fase 1: Recolección de información y caracterización de cargas

La primera fase tiene por objetivo construir el conjunto de datos etiquetado que sustentará el entrenamiento del clasificador, junto con el instrumento y el protocolo que garantizan su validez. Se describe a continuación el procedimiento completo.

2.3.1. Verificación previa de la plataforma

Antes de ejecutar cualquier medición se aplica una batería de verificaciones automáticas sobre el nodo, orientada a confirmar que las condiciones supuestas por el diseño experimental se cumplen efectivamente. Estas verificaciones se agrupan en cinco familias: capacidades del entorno (estado de las tecnologías de aceleración de frecuencia, afinidad NUMA de los núcleos asignados, política de multihilo declarada, dominio real de frecuencia, número de contadores simultáneos disponibles); integridad del catálogo (existencia y permisos de ejecución de cada binario, correspondencia de su suma de verificación, validez del criterio de éxito declarado, suficiencia de memoria); disponibilidad de instrumentación (dominios energéticos solicitados frente a los presentes, corrección del manejo de desbordamiento); condiciones de almacenamiento y presupuesto; y, cuando la campaña contempla control de frecuencia, permisos efectivos de escritura y cobertura del dominio.

Las verificaciones se clasifican en bloqueantes y no bloqueantes. Una verificación bloqueante que no se satisface detiene la campaña antes de tocar el sistema, bajo el principio de que es preferible no producir datos a producir datos cuya validez no puede sostenerse. Este mecanismo se ejecuta de forma automática como parte del comando que lanza la campaña, y no como un paso manual previo, de modo que no exista la posibilidad de iniciar una medición sin haberlo aplicado.

2.3.2. Catálogo de cargas de trabajo

El catálogo experimental se compone de cargas de dos roles distintos. Las cargas de calibración sirven para determinar empíricamente los techos del modelo Roofline sobre la plataforma: un microbenchmark de ancho de banda de memoria, cuyo conjunto de trabajo excede ampliamente la capacidad de la caché de último nivel para garantizar que el tráfico alcance la memoria principal, proporciona la estimación de BW_{pico} ; y un microbenchmark de

densidad aritmética sobre datos residentes en caché proporciona la estimación de P_{pico} . Las cargas de conjunto de datos son los benchmarks cuyo comportamiento se pretende caracterizar.

La Tabla 2.3 resume la composición del catálogo. Todas las cargas de conjunto de datos provienen de suites de benchmarks científicos de amplia adopción, se emplean sin modificar su código fuente y se compilan sobre la plataforma experimental con las opciones por defecto de cada suite, registrando la suma de verificación del binario resultante. Este registro cumple una función metodológica precisa: permite confirmar, antes de cada corrida individual, que el binario ejecutado es exactamente el mismo que se caracterizó, descartando la posibilidad de que una recompilación silenciosa altere las condiciones a mitad de campaña. Dado que un mismo código fuente compilado con cadenas de herramientas distintas produce binarios funcionalmente equivalentes pero con sumas de verificación diferentes, este registro se mantiene asociado a la plataforma en la que se realizó la compilación.

Tabla 2.3: Composición del catálogo experimental de cargas de trabajo.

Rol	Cantidad	Propósito
Calibración	2	Estimación empírica de BW_{pico} (ancho de banda sostenido de memoria) y de P_{pico} (densidad aritmética sobre datos en caché).
Conjunto de datos	6	Kernels de una suite de benchmarks científicos paralelos, seleccionados por reportar su trabajo en operaciones de punto flotante y cubrir distintos patrones de acceso a memoria (resolución de sistemas por métodos implícitos, multigrid, gradiente conjugado, transformada rápida de Fourier).
Conjunto de datos	1	Multiplicación densa de matrices sobre una biblioteca de álgebra lineal optimizada, como referencia de régimen intensivo en cómputo.

2.3.3. Calibración y criterio de etiquetado

La calibración se ejecuta al inicio de cada campaña, sobre la misma asignación de núcleos y con la misma política de afinidad que se emplea después en las corridas de caracterización, de modo que los techos obtenidos correspondan a la configuración realmente utilizada. A partir

de los valores medidos se calcula el punto de inflexión según la ecuación (1.3). Como control de validez, los valores obtenidos se contrastan contra una referencia empírica declarada en el manifiesto de la campaña, y una discrepancia superior a una tolerancia establecida detiene la ejecución: esta verificación protege frente a calibraciones realizadas bajo condiciones anómalas que pasarían inadvertidas si el valor resultante se aceptara sin contraste.

Sobre esta base se define el etiquetado. Cada ventana de muestreo recibe una etiqueta derivada exclusivamente de la comparación entre su intensidad operacional observada y el punto de inflexión calibrado, sin intervención de juicio manual. Adicionalmente, cada carga del catálogo declara una expectativa cualitativa de régimen, proveniente de la literatura sobre la suite correspondiente. Es importante subrayar que ambas anotaciones cumplen funciones distintas y no deben confundirse: la etiqueta derivada del modelo constituye la variable objetivo del problema de clasificación, mientras que la expectativa declarada actúa únicamente como control de coherencia. La correspondencia entre ambas, evaluada de forma agregada por carga, permite detectar errores sistemáticos del instrumento, pero la expectativa nunca sustituye ni corrige la etiqueta medida.

Como control adicional de estabilidad, cada campaña incluye repeticiones de una carga de referencia, sobre las cuales se calcula la dispersión relativa de las métricas de eficiencia. Una dispersión elevada indica que las condiciones de medición no son suficientemente estables como para sostener comparaciones finas entre corridas, y se registra como advertencia asociada a la campaña.

2.3.4. Caracterización de intensidad operacional y calibración Roofline en GPU

En el dominio CPU, la intensidad operacional se calcula por ventana a partir de la telemetría muestreada durante la corrida (sección 1.1.2), incluyendo la medición directa de FLOPs mediante `FP_ARITH_INST_RETIRED` descrita en esa misma sección. El dominio GPU sigue un procedimiento distinto, más cercano en espíritu a la calibración de los techos del modelo Roofline que a un muestreo continuo por ventana: la intensidad operacional de cada carga se determina mediante perfilado directo con un depurador de instrucciones a nivel de kernel (NVIDIA Nsight Compute) [38], que expone contadores de hardware capaces de contabilizar, por cada lanzamiento de kernel, tanto el tráfico total hacia memoria del dispositivo como el número de instrucciones aritméticas de punto flotante efectivamente ejecutadas, discriminadas por tipo de operación (suma, multiplicación y multiplicación–suma fusionada) y por precisión (simple o doble). A partir de estos conteos, la intensidad operacional de la carga se obtiene de forma directa como el cociente entre el total de operaciones de punto flotante

— contando cada multiplicación–suma fusionada como dos operaciones — y el total de bytes transferidos, agregados sobre un número fijo de lanzamientos consecutivos del kernel.

Esta medición se realiza una única vez por carga, de forma independiente de la campaña de adquisición de telemetría, y su resultado se declara como una propiedad fija del catálogo, análoga a la suma de verificación del binario: al igual que ocurre en CPU, la intensidad operacional de un kernel es una propiedad de su algoritmo y del tamaño del problema que resuelve, no del estado de frecuencia bajo el cual se ejecuta, por lo que no es necesario — ni válido, dado el costo de intrusión del perfilado detallado sobre el tiempo de ejecución — repetir esta medición dentro de cada corrida de la campaña principal. Esta vía constituye, igual que en CPU, una medición directa de ambos términos de la ecuación (1.4), aunque agregada por lanzamiento de kernel en vez de por ventana temporal, dada la naturaleza distinta del perfilado de GPU frente al muestreo periódico de contadores de CPU.

Conviene señalar explícitamente por qué el mecanismo de perfilado en GPU no hereda la restricción de presupuesto de contadores físicos simultáneos discutida para CPU en la sección 1.1.3, ni el riesgo de multiplexación asociado a ella. Un contador de PMU de CPU, abierto mediante `perf_event_open`, se muestrea en vivo mientras la carga corre una sola vez; cuando el número de eventos solicitados excede el número de contadores físicos disponibles, el kernel reparte el tiempo de esa única ejecución entre ellos y extrapola cada lectura mediante el factor de escalado tiempo-habilitado/tiempo-contando (sección 1.1.3), un procedimiento que asume una tasa de evento uniforme durante todo el intervalo y que resulta especialmente riesgoso cuando la ventana de observación es corta frente al período de rotación del kernel. Nsight Compute, en cambio, no muestrea en vivo una única ejecución: cuando las métricas solicitadas exceden lo que el hardware de contadores de la GPU puede capturar en una sola pasada, la herramienta *reproduce* el lanzamiento del kernel completo tantas veces como sea necesario, una pasada por cada subconjunto de métricas, bajo la condición de que el kernel sea determinista frente a sus mismas entradas. Cada métrica se obtiene así de una ejecución íntegra del kernel, no de una fracción de tiempo compartida con otras métricas, por lo que no existe una extrapolación lineal equivalente a la de CPU ni una ventana de observación corta que pueda quedar a mitad de una rotación. La precisión aritmética (simple o doble) se discrimina, en consecuencia, sin ningún compromiso de presupuesto: da igual cuántas métricas o precisiones se soliciten, el costo se traduce en más pasadas de perfilado, no en una restricción de cuántas pueden coexistir — un costo asumible precisamente porque, como se explicó arriba, esta medición ocurre una única vez por carga, fuera de la campaña de adquisición de telemetría en tiempo real.

La calibración de los techos del modelo Roofline en GPU sigue el mismo principio general

descrito para CPU — microbenchmarks dedicados a saturar por separado cada límite, bajo las condiciones reales de la plataforma —, con dos particularidades propias del dispositivo. Primera, dado que las GPU consideradas ejecutan operaciones de simple y doble precisión a tasas pico sustancialmente distintas, P_{pico} se calibra por separado para cada precisión, y el punto de inflexión de la ecuación (1.3) se calcula igualmente por separado, de modo que una carga expresada en una precisión se compara exclusivamente contra el techo correspondiente a esa misma precisión. Segunda, la carga empleada para calibrar P_{pico} se implementó como un microbenchmark propio de multiplicación–suma fusionada sobre datos residentes en registros, en lugar de recurrir a una biblioteca de álgebra lineal optimizada de terceros: una biblioteca de este tipo puede seleccionar internamente, sin que el usuario lo solicite explícitamente, una ruta de cómputo acelerada por hardware especializado no representativo de la aritmética general de punto flotante del dispositivo, lo que produciría un techo de cómputo inflado frente al alcanzable por las cargas del catálogo que no emplean dicha ruta, y en consecuencia clasificaría erróneamente como *memory-bound* cargas que, medidas contra el techo de aritmética vainilla correspondiente a su precisión, resultan *compute-bound*. Este riesgo es formalmente equivalente al ya discutido en la sección 1.1.7 para el caso de una carga que cuantifica su trabajo en una unidad distinta de FLOPs: en ambos casos, comparar magnitudes que no están expresadas en el mismo referente invalida la clasificación resultante, aunque el error no se manifieste como una falla explícita del instrumento.

2.3.5. Dimensionamiento de las cargas GPU y verificación de invarianza de la intensidad operacional

Puesto que la intensidad operacional de cada carga GPU se fija mediante la medición directa descrita en la sección 2.3.4, cualquier ajuste posterior de los parámetros de ejecución de una carga — por ejemplo, para alcanzar una duración de corrida suficiente para los fines descritos en la sección 2.3.7 — exige distinguir explícitamente entre dos categorías de parámetros según su efecto sobre dicha medición.

La primera categoría comprende los parámetros que repiten el mismo patrón de cómputo un mayor número de veces sin alterar el tamaño del problema resuelto en cada repetición — un número de iteraciones de un mismo esquema numérico, o la duración simulada de un proceso cuyo paso de integración permanece fijo —. Un ajuste de esta naturaleza no puede alterar la intensidad operacional del kernel, dado que cada repetición ejecuta exactamente el mismo balance entre trabajo aritmético y tráfico de memoria que la anterior; en consecuencia, este tipo de ajuste puede aplicarse sin necesidad de repetir la medición de la sección 2.3.4. La segunda categoría comprende los parámetros que alteran el tamaño real del problema

— la geometría de una malla, el número de elementos de una simulación de N-cuerpos, o la resolución de una imagen —, para los cuales dicha invarianza no puede asumirse: un problema de mayor tamaño puede modificar la proporción entre trabajo aritmético y tráfico de memoria, entre otras razones porque el conjunto de datos activo deja de residir en un nivel de la jerarquía de caché que sí alcanzaba a contener al problema original, incrementando el tráfico efectivo hacia memoria principal de forma no proporcional al aumento del trabajo aritmético. Por ello, todo ajuste de la segunda categoría se trata como una modificación que invalida la caracterización previa de la carga, y se somete obligatoriamente a una nueva medición mediante el procedimiento de la sección 2.3.4 antes de aceptar cualquier dato adquirido bajo la nueva configuración; cuando dicha remediación confirma un cambio material en la intensidad operacional, se verifica adicionalmente que la carga conserve su clasificación cualitativa frente al punto de inflexión vigente, dado que un cambio de magnitud no implica necesariamente un cambio de régimen.

Cuando una carga no dispone de un parámetro de la primera categoría — por ejemplo, porque su implementación no contempla una noción de repetición interna del mismo cómputo —, la única vía disponible para extender su duración es un parámetro de la segunda categoría, con el costo de verificación que ello implica; y cuando, adicionalmente, el modelo de ejecución del dispositivo impone un límite estructural sobre dicho parámetro — por ejemplo, un límite documentado sobre el número de unidades de ejecución concurrentes direccionables por un único lanzamiento de kernel —, la duración máxima alcanzable para esa carga queda acotada por dicho límite de hardware, con independencia de cualquier criterio estadístico de suficiencia de muestreo. En tales casos, la imposibilidad de extender la corrida más allá del límite estructural del dispositivo se declara como una limitación del instrumento específica de esa carga, y no se fuerza una extensión que comprometería la validez de la caracterización ya obtenida.

2.3.6. Matriz experimental y control de sesgos

La matriz experimental se define como el producto cartesiano entre las cargas del catálogo, los estados de frecuencia y las repeticiones. Los estados de frecuencia previstos se describen en la Tabla 2.4: además del punto de operación nativo, que sirve como referencia y se obtiene sin intervenir el gobernador del sistema, se contemplan cinco niveles distribuidos uniformemente sobre el rango soportado por la plataforma. Los niveles fijos se expresan como fracciones del rango real detectado en tiempo de ejecución, y no como valores absolutos de frecuencia, con el fin de que la misma definición de campaña sea aplicable a plataformas con rangos distintos.

Tabla 2.4: Estados de frecuencia de la matriz experimental.

Nivel	Definición	Propósito
REF	Gobernador nativo, sin control manual	Línea base de comparación
F0	Extremo superior del rango	Límite de rendimiento y consumo
F1	75 % del rango	Punto intermedio alto
F2	50 % del rango	Punto intermedio central
F3	25 % del rango	Punto intermedio bajo
F4	Extremo inferior del rango	Límite inferior

Para reducir el sesgo experimental se adoptan cuatro medidas. Primera, cada combinación se ejecuta en múltiples repeticiones independientes, entendidas como relanzamientos completos del binario y no como iteraciones internas, de modo que cada repetición parta de un estado de caché y de predictores equivalente. Segunda, el orden de ejecución de las combinaciones se aleatoriza a partir de una semilla registrada en el manifiesto, con el fin de evitar que una deriva temporal — por ejemplo, térmica — se confunda con el efecto de la carga o de la frecuencia; el registro de la semilla preserva la reproducibilidad del orden. Tercera, las condiciones de afinidad, número de hilos y asignación de núcleos se fijan explícitamente y de forma idéntica para todas las corridas, en lugar de heredarse del entorno. Cuarta, al concluir la campaña se verifica que el estado de frecuencia del sistema haya sido restaurado a su configuración original, y este mismo mecanismo de restauración se registra para ejecutarse también ante una terminación anómala.

2.3.7. Post-procesamiento y control de calidad de los datos

Las muestras crudas se transforman en el conjunto de datos final mediante un procedimiento de diferenciación entre lecturas consecutivas, conforme a lo descrito en la sección 1.1.3. Cada fila resultante representa una ventana de ejecución e incorpora, además de las métricas derivadas, un conjunto de campos de trazabilidad que permiten reconstruir su procedencia: identificador de corrida, carga, nodo, estado de frecuencia solicitado y observado, referencias a los archivos de calibración empleados y suma de verificación del binario ejecutado.

Un elemento central de este procedimiento es que ninguna observación se descarta silenciosamente. Cada ventana recibe una anotación de calidad que describe su condición, según la taxonomía de la Tabla 2.5, y se conserva en el conjunto de datos con dicha anotación. Esta decisión responde a un criterio de transparencia: el filtrado posterior es una decisión del análisis, no del instrumento, y la proporción de ventanas en cada categoría constituye en sí

misma un indicador de la calidad de la campaña.

Entre estas categorías, la exclusión por calentamiento delimita explícitamente, para cada carga del catálogo, un intervalo inicial de la corrida declarado como transitorio de arranque, correspondiente al período en que la ejecución aún no alcanza un comportamiento estacionario — por ejemplo, mientras se completan la asignación de memoria, la creación de hilos y la sincronización inicial de la carga. Las ventanas cuya marca temporal cae dentro de ese intervalo se anotan como excluidas por calentamiento y no se emplean en el análisis, de modo que la región efectivamente caracterizada corresponda a la fase repetitiva del kernel y no a su arranque. A diferencia de otras anotaciones de la Tabla 2.5, que se derivan de una condición verificable en la propia ventana, la duración de este intervalo no se fija arbitrariamente por carga ni se estima a partir de una expectativa general de arranque: se determina mediante un procedimiento estadístico aplicado sobre la serie de telemetría de cada corrida, descrito a continuación.

Un aspecto favorable para la aplicación de este procedimiento es que la duración del calentamiento nunca condiciona la adquisición: el intervalo declarado se emplea exclusivamente como criterio de filtrado durante el post-procesamiento, y no se comunica en ningún momento al ejecutor de la capa C++, de modo que toda corrida ya adquirida contiene, desde el primer instante, la totalidad del transitorio de arranque real, con independencia del valor de calentamiento vigente al momento de adquirirla. Esta propiedad permite recalcular y ajustar el criterio de calentamiento de forma retroactiva sobre corridas ya realizadas, sin necesidad de una nueva adquisición.

El procedimiento se desarrolla en dos etapas, aplicadas sobre una señal representativa del estado de carga del dispositivo — instrucciones por ciclo en el caso de CPU, utilización del dispositivo reportada por la interfaz de gestión de GPU en el caso de GPU —, calculada sobre ventanas móviles consecutivas de tamaño fijo.

En la primera etapa se aplica un criterio de estabilidad basado en el coeficiente de variación (CV %) de dicha señal, siguiendo el principio de evaluación estadísticamente rigurosa de desempeño propuesto por Georges et al. [36]: se declara alcanzado el fin del transitorio de arranque en el primer instante a partir del cual el CV % se mantiene por debajo de un umbral en dos ventanas consecutivas — exigencia de dos ventanas, y no de una sola, para no confundir el fin del arranque con una fluctuación transitoria dentro de una fase todavía inestable, ni con un cambio de fase legítimo del propio kernel que ocurra más adelante en la corrida —, y al punto detectado se le aplica un margen de seguridad multiplicativo antes de declararlo como intervalo de calentamiento definitivo. Una limitación de este criterio, identi-

ficada durante su verificación empírica sobre la señal de utilización de GPU, merece señalarse explícitamente: dado que el coeficiente de variación se define como el cociente entre la desviación estándar y la media, una ventana cuya señal permanece en cero satisface trivialmente el umbral, de modo que el criterio, aplicado sin más, puede declarar estable el reposo inicial del dispositivo — previo a que exista actividad real — en lugar de la fase de carga sostenida que se pretende delimitar, precisamente el resultado opuesto al buscado. Este modo de falla se corrige exigiendo, adicionalmente al umbral de CV %, que la media de la ventana supere un piso mínimo de actividad antes de aceptar la ventana como estable.

En la segunda etapa, reservada para las cargas donde el criterio de CV % no logra identificar ningún punto que satisfaga la condición anterior — un modo de falla conocido de este tipo de heurística de umbral fijo —, se aplica como respaldo un método de detección de puntos de cambio (*change point detection*) sobre la misma señal: en lugar de asumir que el estado estable es plano dentro de un umbral, el método busca recursivamente la partición de la serie que más reduce la suma de errores cuadráticos dentro de cada segmento resultante, en la línea de los métodos empleados para caracterizar estadísticamente el comportamiento de arranque de máquinas virtuales [37]. Cuando la serie presenta más de un punto de cambio — por ejemplo, una fluctuación breve y espuria previa al inicio real de la carga sostenida —, se descarta el criterio de aceptar sin más el primer punto detectado, y se selecciona en su lugar el primer segmento cuya media alcanza una fracción declarada de la meseta de mayor carga observada en toda la corrida, de modo que una fluctuación que no llega a sostenerse no se confunda con el fin genuino del arranque.

Como verificación final, antes de aceptar el resultado de cualquiera de las dos etapas, la transición detectada se contrasta contra una señal de referencia independiente de la empleada para la detección — la potencia reportada por el dispositivo —: una transición genuina de régimen de reposo a régimen de carga sostenida debe manifestarse también como un salto identificable en la potencia consumida, y no únicamente en la señal primaria de detección. Cuando ninguna de las dos etapas anteriores encuentra, en la señal de potencia, una transición identificable a lo largo de toda la corrida — situación observada en cargas cuyas unidades individuales de trabajo resultan demasiado breves para que la cadencia de muestreo del dispositivo resuelva su actividad, con independencia de cuánto se alargue la duración total de la corrida conforme al criterio de la sección 2.3.5 —, se concluye que la señal disponible no sostiene una medición confiable del intervalo de calentamiento para esa carga específica. En ese caso, el intervalo se conserva en su valor por defecto y la imposibilidad de medirlo se documenta explícitamente como evidencia negativa, en lugar de forzar un valor no respaldado por la medición.

Tabla 2.5: Taxonomía de anotaciones de calidad por ventana de muestreo.

Anotación	Condición
Válida	La ventana satisface todos los criterios de calidad.
Sin diferencia previa	Primera muestra de la corrida; carece de lectura anterior contra la cual diferenciar.
Excluida por calentamiento	La ventana pertenece al intervalo inicial declarado como transitorio de arranque.
Contadores degradados	Diferencia negativa, valor ausente, o fracción de tiempo contando inferior al umbral admitido.
Intensidad indefinida	No fue posible calcular la intensidad operacional, por ausencia de trabajo reportado o de tráfico observable.
Energía inválida	La lectura energética correspondiente no es válida en una campaña que la requiere.
Sin lectura de frecuencia	No se dispone de la frecuencia efectivamente observada durante la ventana.

De forma complementaria, cada corrida completa se somete a una validación posterior que examina su integridad global: ausencia de pérdidas en la estructura de muestreo, correspondencia de la suma de verificación del binario, cumplimiento del criterio de éxito propio de la carga y unicidad del identificador de corrida. Una corrida que no supere esta validación se registra como rechazada, conservando sus artefactos para inspección, en lugar de eliminarse.

2.3.8. Medición directa de FLOPs por ventana y su validación empírica

El diseño original de este instrumento estimaba los FLOPs de cada ventana mediante el prorrateo de la ecuación (1.6), bajo el supuesto de que la densidad de operaciones de punto flotante por instrucción retirada, ρ_i , se mantiene aproximadamente constante e igual a la densidad promedio de toda la corrida, ρ_{global} . Dicho supuesto no se dio por válido sin verificación. Antes de adoptarlo como definitivo, se evaluó si la plataforma experimental permitía sustituirlo por una medición directa, siguiendo el mismo criterio de no asumir capacidades del hardware sin confirmarlas empíricamente que rige el resto de este instrumento (secciones 1.1.5 y 1.1.3). Esta sección documenta ese procedimiento de verificación y su resultado.

Diseño de la verificación. Se construyó un instrumento de prueba independiente del harness de producción, con el mismo protocolo de atribución por PID e `inherit=1` descrito en la sección 1.1.5, capaz de abrir simultáneamente los diez contadores del presupuesto completo de la plataforma: los seis ya establecidos (sección 1.1.3) más los cuatro sub-eventos de `FP_ARITH_INST_RETIRED` de la ecuación (1.5). Este instrumento se ejecutó contra dos kernels de régimen opuesto del catálogo experimental: uno intensivo en cómputo (multiplicación de matrices densas, con un volumen de trabajo conocido analíticamente) y uno intensivo en memoria (un kernel multi-hilo de la suite NAS Parallel Benchmarks), en ambos casos bajo la afinidad de núcleos real de la campaña.

Resultados. Para el kernel de cómputo denso, cuyo total de FLOPs se conoce exactamente por construcción ($2 \cdot \text{iteraciones} \cdot n^3$), la ecuación (1.5) reprodujo ese total con un error relativo de 0.29 % sin afinidad fijada y de 0.30 % bajo la afinidad real de campaña. Para el kernel intensivo en memoria, el contraste se hizo contra el total de FLOPs que el propio kernel reporta al finalizar, arrojando un error relativo de 7.48 %; este valor es mayor que el del primer kernel, pero es consistente con una causa identificada y ajena al contador: el tiempo que dicho kernel cronometra internamente cubre únicamente su ciclo de cómputo principal y excluye la fase final de verificación del resultado, la cual también ejecuta operaciones de punto flotante que el contador de hardware sí registra por abarcar el proceso completo. En ambos casos, los diez contadores abrieron simultáneamente sin evidencia de multiplexación (razón tiempo-contando sobre tiempo-habilitado igual a 1 en los diez), confirmando que el presupuesto físico de la plataforma sostiene esta combinación completa sin degradación de la medida.

Confirmación a escala de campaña. Con la verificación anterior favorable, la medición directa se integró al harness de producción como fuente principal de FLOPs por ventana, reemplazando al prorrato salvo como respaldo (ecuación (1.6)), y se ejecutó una campaña real completa — siete kernels del catálogo, seis estados de frecuencia, tres repeticiones por combinación, 66 corridas aceptadas o rechazadas por los criterios de la sección 2.3.7 y siguientes. Sobre aproximadamente 1.29 millones de ventanas con delta válido en esa campaña, el 100 % obtuvo su valor de FLOPs mediante la ecuación (1.5); el prorrato de respaldo no se activó en ninguna, y ninguna ventana resultó con los contadores de punto flotante degradados. Este resultado, a una escala varios órdenes de magnitud mayor que la verificación inicial de dos kernels, respalda que la disponibilidad y estabilidad del encoding no es un caso aislado favorable, sino una propiedad sostenida de la combinación plataforma-instrumento descrita en este capítulo.

Alcance de esta validación. Los resultados anteriores se verificaron específicamente para la microarquitectura Ice Lake-SP de la plataforma experimental (Tabla 2.1), gateados por familia y modelo de procesador en el mismo mecanismo que protege los demás eventos crudos de este instrumento (sección 1.1.3); no se extrapolan a otro hardware sin repetir este procedimiento. Tampoco se reclama que el error relativo observado (0.29–7.48 %) sea una cota universal: ambos valores dependen del kernel evaluado y, en el segundo caso, de una particularidad de cómo ese kernel específico delimita su propio tiempo de ejecución. La ecuación (1.6) se conserva activa en el instrumento exclusivamente como mecanismo de respaldo para un nodo o una corrida donde este encoding no se haya validado.

2.3.9. Reproducibilidad

Toda campaña queda definida por un manifiesto declarativo bajo control de versiones, que especifica las cargas, los estados de frecuencia, el número de repeticiones, el período de muestreo, la asignación de núcleos, la semilla de aleatorización y las referencias de calibración. Junto con los datos, cada campaña persiste el perfil de capacidades detectado en el nodo, los resultados de calibración y los metadatos de cada corrida. El instrumento de medición, el orquestador y las definiciones de campaña se mantienen en un repositorio público, de modo que la campaña sea reproducible a partir de los artefactos publicados.

Como verificación adicional de consistencia, toda vez que se modifica un parámetro de ejecución de una carga del catálogo — por ejemplo, a raíz de un ajuste realizado conforme a lo descrito en la sección 2.3.5 — se ejecuta una corrida completa de la campaña que incluya dicha carga, contrastando que la etiqueta derivada, la intensidad operacional y el punto de inflexión empleado resulten idénticos entre las distintas repeticiones independientes de esa misma carga. Esta comparación no sustituye la verificación de calidad por ventana descrita en la Tabla 2.5, sino que actúa como una verificación de nivel superior: una discrepancia entre repeticiones nominalmente idénticas indicaría una fuente de no determinismo en el instrumento o en la propia carga, y no únicamente un artefacto puntual de una corrida concreta.

2.4. Fase 2: Desarrollo del modelo de aprendizaje automático

En esta fase se desarrollan las técnicas de análisis orientadas a la construcción de un modelo predictivo capaz de identificar el comportamiento de la carga de trabajo en tiempo de ejecución. Inicialmente se realizará un proceso de preprocesamiento que incluye la limpieza,

la normalización y la selección de características relevantes, partiendo del conjunto de datos producido en la Fase 1 y aplicando sobre él los criterios de filtrado que se estimen adecuados a partir de las anotaciones de calidad descritas en la Tabla 2.5.

El problema se formula como una tarea de clasificación supervisada binaria, donde el modelo recibe como entrada un vector de telemetría correspondiente a una ventana de ejecución y produce como salida la etiqueta de régimen. Dado que el sistema debe operar en tiempo de ejecución, se prioriza el uso de modelos de baja complejidad computacional, entre ellos árboles de decisión, bosques aleatorios, regresión logística y ensambles potenciados. Durante esta fase se entrenarán y evaluarán distintos modelos candidatos, comparados mediante métricas de clasificación y mediante la latencia de inferencia medida.

Un criterio metodológico relevante en esta fase es la estrategia de partición del conjunto de datos. Dado que las ventanas provenientes de una misma corrida no son observaciones independientes entre sí, una partición aleatoria a nivel de ventana produciría una estimación optimista de la capacidad de generalización, al permitir que ventanas contiguas de la misma ejecución aparezcan simultáneamente en entrenamiento y prueba. Por ello, la partición se realizará a nivel de corrida o de carga, de modo que la evaluación mida la capacidad del modelo para generalizar hacia ejecuciones no observadas. A partir de este análisis se seleccionará el modelo que represente el mejor compromiso entre desempeño predictivo y costo de inferencia, y se serializará para su integración en el sistema de control.

2.5. Fase 3: Implementación del agente de control DVFS

La tercera fase corresponde al desarrollo del agente de control en espacio de usuario, materializado como un servicio en segundo plano encargado de monitorear el estado del hardware y aplicar decisiones de control basadas en el modelo entrenado.

El funcionamiento se basa en un ciclo iterativo en el cual se capturan las métricas actuales del sistema, se construye el vector de entrada del modelo, se ejecuta la inferencia y, a partir de la predicción, se determina y aplica la configuración de frecuencia. La política de control será discreta: no resolverá un problema continuo de optimización, sino que seleccionará un estado dentro del conjunto de configuraciones soportadas por el hardware. La decisión se tomará de forma proactiva, guiada por el régimen inferido y no únicamente por métricas reactivas de utilización.

El diseño de esta fase incorpora dos consideraciones derivadas del marco conceptual. La primera es que el mecanismo de actuación debe adaptarse al controlador de frecuencia activo

en la plataforma, dado que la vía para fijar un punto operativo difiere según se trate de un controlador que expone un gobernador de espacio de usuario o de uno que solo admite la restricción del rango permitido. La segunda es que la frecuencia de decisión debe acotarse considerando la latencia de conmutación: una política que solicite cambios a un ritmo superior al que el sistema puede materializar no solo no obtendrá el beneficio esperado, sino que incurrirá en el costo de las transiciones sin amortizarlo [18, 27]. En consecuencia, el período de decisión del agente se tratará como un parámetro de diseño a determinar empíricamente, y no como una constante fijada a priori.

2.6. Fase 4: Validación experimental y análisis de resultados

La fase final evalúa empíricamente el impacto del sistema propuesto sobre el consumo energético y el rendimiento, en concordancia con el cuarto objetivo específico. Se ejecutarán las cargas seleccionadas bajo escenarios de referencia: gobernador nativo del sistema operativo, configuración de frecuencia fija de alto rendimiento, y ejecución orquestada por el agente propuesto. En cada ejecución se medirán el tiempo total de ejecución, la energía consumida, la potencia media, el EDP y el *overhead* atribuible al agente.

Dado que el objetivo no es únicamente reportar métricas sino establecer si las diferencias observadas son estadísticamente defendibles, los resultados se analizarán mediante técnicas estadísticas seleccionadas según la distribución y la homogeneidad de varianzas de los datos, reportando medidas de dispersión y tamaños de efecto junto con las pruebas de significancia.

2.7. Aspectos éticos

La presente investigación no involucra participantes humanos, datos personales ni información sensible, por lo que no requiere aprobación de un comité de ética en investigación con seres humanos. No obstante, se adoptan las siguientes consideraciones éticas, pertinentes a la naturaleza del trabajo.

En cuanto al uso responsable de infraestructura compartida, los experimentos se ejecutan sobre recursos de cómputo institucionales de acceso compartido. En consecuencia, se adopta como principio que ninguna acción del experimento debe degradar las condiciones de ejecución de terceros: las mediciones se realizan mediante asignación exclusiva otorgada por el gestor de recursos, y toda actuación sobre la frecuencia se restringe a los núcleos efectivamente asignados, verificando previamente que el dominio de frecuencia afectado no exceda

dicha asignación, de acuerdo con lo expuesto en la sección 1.1.6. Del mismo modo, el estado de frecuencia del sistema se restaura al concluir la campaña, incluso ante terminaciones anómalas.

En cuanto a la integridad de los datos y de los resultados, se adopta como criterio que ninguna observación se descarte de forma silenciosa: las ventanas que no satisfacen los criterios de calidad se conservan con su anotación correspondiente, y las corridas rechazadas preservan sus artefactos. Las limitaciones conocidas del instrumento — en particular, el carácter aproximado de la estimación de bytes movidos discutido en la sección 1.1.2 — se declaran explícitamente junto con los resultados que dependen de ellas, en lugar de omitirse.

En cuanto al uso de software de terceros, las suites de benchmarks empleadas se utilizan sin modificación de su código fuente y respetando sus condiciones de distribución, y su procedencia se registra mediante la suma de verificación de los archivos originales obtenidos. Finalmente, en cuanto a transparencia y reproducibilidad, el instrumento de medición, el orquestador y las definiciones de campaña se publican en un repositorio de acceso público, junto con la documentación de las decisiones metodológicas adoptadas durante el desarrollo, de modo que los resultados puedan ser verificados o refutados por terceros.

Capítulo 3

Resultados

3.1. Descripción del conjunto de datos experimental

Capítulo 4

Discusión

Capítulo 5

Conclusiones

5.1. Limitaciones

5.2. Trabajo Futuro

Bibliografía

- [1] O. S. Simsek, J.-G. Piccinali, and F. M. Ciorba, “Increasing Energy Efficiency of Astrophysics Simulations Through GPU Frequency Scaling,” in *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, Atlanta, GA, USA, 2024, pp. 1826–1834. doi: 10.1109/SCW63240.2024.00229.
- [2] M. T. Costa et al., “Characterizing the Impact of GPU Power Management on an Exascale System,” in *Proc. SC ’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops ’25)*, New York, NY, USA, 2025, pp. 1524–1533. doi: 10.1145/3731599.3767702.
- [3] F. Antici, A. Bartolini, Z. Kiziltan, O. Babaoglu, and Y. Kodama, “MCBound: An Online Framework to Characterize and Classify Memory/Compute-bound HPC Jobs,” in *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, Atlanta, GA, USA, 2024, pp. 1–15. doi: 10.1109/SC41406.2024.00062.
- [4] E. Calore, A. Gabbana, S. F. Schifano, and R. Tripiccione, “Evaluation of DVFS techniques on modern HPC processors and accelerators for energy-aware applications,” *Concurrency Comput. Pract. Exper.*, vol. 29, no. 12, p. e4143, 2017. doi: 10.1002/cpe.4143.
- [5] G. Ali, M. Side, S. Bhalachandra, N. J. Wright, and Y. Chen, “An automated and portable method for selecting an optimal GPU frequency,” *Future Generation Computer Systems*, vol. 149, pp. 71–88, Dec. 2023. doi: 10.1016/j.future.2023.07.011.
- [6] R. Gonzalez, B. M. Gordon, and M. A. Horowitz, “Supply and threshold voltage scaling for low power CMOS,” *IEEE J. Solid-State Circuits*, vol. 32, no. 8, pp. 1210–1216, Aug. 1997. doi: 10.1109/4.604018.
- [7] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009. doi: 10.1145/1498765.1498785.

- [8] T.-Y. Liu, J. Guo, and B. Huang, “Efficient Cross-Platform Multiplexing of Hardware Performance Counters via Adaptive Grouping,” *ACM Trans. Archit. Code Optim.*, vol. 21, no. 1, pp. 1–26, Mar. 2024. doi: 10.1145/3629525.
- [9] S.-K. Shekoffteh et al., “Metric selection for GPU kernel classification,” *ACM Trans. Archit. Code Optim.*, vol. 15, no. 4, pp. 1–27, Jan. 2019. doi: 10.1145/3295690.
- [10] L. Littman and T. Deakin, “Classifying Performance Bounds Using Machine Learning,” in *Proc. SC25: Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2025.
- [11] B. Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2020.
- [12] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. San Francisco, CA, USA: No Starch Press, 2010.
- [13] S. McCarty, “Architecting Containers Part 1: Why Understanding User Space vs. Kernel Space Matters,” Red Hat Blog, Jul. 29, 2015. [Online]. Available: <https://www.redhat.com/en/blog/architecting-containers-part-1-why-understanding-user-space-vs-kernel-space-matters> [Accessed: Apr. 3, 2025].
- [14] V. M. Weaver, “Self-monitoring overhead of the Linux perf event performance counter interface,” in *Proc. IEEE Int. Symp. Performance Analysis of Systems and Software (ISPASS)*, 2015, pp. 102–111.
- [15] H. David, E. Gorbato, U. R. Hanebutte, R. Khanna, and C. Le, “RAPL: Memory power estimation and capping,” in *Proc. 16th ACM/IEEE Int. Symp. Low Power Electron. Des. (ISLPED)*, Aug. 2010, pp. 189–194. doi: 10.1145/1840845.1840883.
- [16] NVIDIA, “NVIDIA Management Library (NVML),” NVIDIA Developer Documentation, 2026. [Online]. Available: <https://docs.nvidia.com/deploy/nvml-api/index.html>. [Accessed: Apr. 3, 2026].
- [17] P. Thamm and U. Leser, “Strategies to measure energy consumption using RAPL during workflow execution on commodity clusters,” arXiv preprint arXiv:2505.09375, 2025.
- [18] R. Hebbar and A. Milenković, “PMU-events-driven DVFS techniques for improving energy efficiency of modern processors,” *ACM Trans. Model. Perform. Eval. Comput. Syst.*, vol. 7, no. 1, pp. 1–31, May 2022. doi: 10.1145/3538645.

- [19] J. Guerreiro, N. Roma, P. Tomás, F. Pratas, L. S. G. Carvalho, and G. Gaydadjiev, “DVFS-aware application classification to improve GPGPUs energy efficiency,” *Parallel Comput.*, vol. 83, pp. 93–117, May 2019. doi: 10.1016/j.parco.2018.02.001.
- [20] ScienceDirect, “Computational Kernel,” Computer Science Topics, 2026. [Online]. Available: <https://www.sciencedirect.com/topics/computer-science/computational-kernel>. [Accessed: Apr. 3, 2026].
- [21] D. Pandey, K. Niwaria, and B. Chourasia, “Machine Learning Algorithms: A Review,” *Int. J. Mach. Learn. Netw. Appl.*, vol. 6, no. 2, pp. 916–922, Jan. 2019. doi: 10.21275/ART20203995.
- [22] IBM, “What is random forest?” IBM Think Topics, 2026. [Online]. Available: <https://www.ibm.com/think/topics/random-forest>. [Accessed: Apr. 3, 2026].
- [23] IBM, “What is logistic regression?” IBM Think Topics, 2026. [Online]. Available: <https://www.ibm.com/think/topics/logistic-regression>. [Accessed: Apr. 3, 2026].
- [24] IBM, “What is XGBoost?” IBM Think Topics, 2026. [Online]. Available: <https://www.ibm.com/think/topics/xgboost>. [Accessed: Apr. 3, 2026].
- [25] J. H. Laros III, K. Pedretti, S. M. Kelly, W. Shu, K. Ferreira, J. Van Dyke, and C. Vaughan, “Energy Delay Product,” in *Energy-Efficient High Performance Computing: Measurement and Tuning*, SpringerBriefs in Computer Science. London, UK: Springer, 2013, ch. 6, pp. 51–55. doi: 10.1007/978-1-4471-4492-2_9.
- [26] Y. Wang, M. Hao, H. He, W. Zhang, Q. Tang, X. Sun, and Z. Wang, “DRLCap: Runtime GPU Frequency Capping with Deep Reinforcement Learning,” *IEEE Trans. Sustain. Comput.*, vol. 9, no. 5, pp. 712–726, Sept.–Oct. 2024. doi: 10.1109/TSUSC.2024.3361596.
- [27] D. Velicka, O. Vysocky, and L. Riha, “Methodology for GPU frequency switching latency measurement,” in *Proc. 2025 IEEE Int. Parallel Distrib. Process. Symp. Workshops (IPDPSW)*, 2025, pp. 830–839. doi: 10.1109/IPDPSW66978.2025.00133.
- [28] Z. Zhang, Z. Wu, J. Liu, and L. Mottola, “SparseDVFS: Sparse-Aware DVFS for Energy-Efficient Edge Inference,” arXiv preprint arXiv:2603.21908, Mar. 23, 2026.
- [29] J. Treibig, G. Hager, and G. Wellein, “LIKWID: A Lightweight Performance-Oriented Tool Suite for x86 Multicore Environments,” in *Proc. 39th Int. Conf. Parallel Processing Workshops (ICPPW)*, San Diego, CA, USA, 2010, pp. 207–216. doi: 10.1109/ICPPW.2010.38.

- [30] G. Hamerly, E. Perelman, J. Sherwood, and B. Calder, “SimPoint 3.0: Faster and More Flexible Program Phase Analysis,” *Journal of Instruction-Level Parallelism*, vol. 7, pp. 1–28, 2005.
- [31] Intel, “CPU Metrics Reference,” Intel VTune Profiler User Guide, 2026. [Online]. Available: <https://www.intel.com/content/www/us/en/docs/vtune-profiler/user-guide/2026-1/cpu-metrics-reference.html>. [Accessed: Aug. 7, 2026].
- [32] Innovative Computing Laboratory, University of Tennessee, “PAPI: Performance Application Programming Interface,” 2026. [Online]. Available: <https://icl.utk.edu/papi/>. [Accessed: Aug. 7, 2026].
- [33] National Energy Research Scientific Computing Center (NERSC), “Roofline Performance Model,” NERSC Documentation, 2026. [Online]. Available: <https://docs.nersc.gov/tools/performance/roofline/>. [Accessed: Aug. 7, 2026].
- [34] V. M. Weaver and S. A. McKee, “Can hardware performance counters be trusted?,” in *Proc. IEEE Int. Symp. Workload Characterization (IISWC)*, Seattle, WA, USA, 2008, pp. 141–150. doi: 10.1109/IISWC.2008.4636099.
- [35] M. A. Laurenzano, M. M. Tikir, L. Carrington, and A. Snavey, “PEBIL: Efficient static binary instrumentation for Linux,” in *Proc. IEEE Int. Symp. Performance Analysis of Systems & Software (ISPASS)*, White Plains, NY, USA, 2010, pp. 175–183. doi: 10.1109/ISPASS.2010.5452024.
- [36] A. Georges, D. Buytaert, and L. Eeckhout, “Statistically rigorous Java performance evaluation,” in *Proc. 22nd Annu. ACM SIGPLAN Conf. Object-Oriented Program. Syst. Appl. (OOPSLA ’07)*, Montreal, QC, Canada, 2007, pp. 57–76. doi: 10.1145/1297027.1297033.
- [37] E. Barrett, C. F. Bolz-Tereick, R. Killick, S. Mount, and L. Tratt, “Virtual Machine Warmup Blows Hot and Cold,” *Proc. ACM Program. Lang.*, vol. 1, no. OOPSLA, art. 52, pp. 1–27, Oct. 2017. doi: 10.1145/3133876.
- [38] NVIDIA, “Nsight Compute,” NVIDIA Developer Documentation, 2026. [Online]. Available: <https://docs.nvidia.com/nsight-compute/>. [Accessed: Aug. 8, 2026].

Repositorio y recursos complementarios

Repositorio público del proyecto:

<https://github.com/AdrianCCRS/hyperion>